



UNIVERSITY OF
LEICESTER

School of Computing and Mathematical Sciences

CO7201 Individual Project Final
Report

Stock Control System

Shubham Hariyale

Email: sh1035@student.le.ac.uk

Word Count: 18,216 (excluding code)
[15/05/2026]

DECLARATION

All sentences or passages quoted in this report, or computer code of any form whatsoever used and/or submitted at any stages, which are taken from other people's work have been specifically acknowledged by clear citation of the source, specifying author, work, date and page(s). Any part of my own written work, or software coding, which is substantially based upon other people's work, is duly accompanied by clear citation of the source, specifying author, work, date and page(s). I understand that failure to do this amounts to plagiarism and will be considered grounds for failure in this module and the degree examination as a whole.

Name: Shubham Hariyale Date: 15/05/2026

Abstract

This project presents the design, implementation and evaluation of a secure web-based Stock Control Management System for managing inventory in a multi-user environment. The system was developed using ASP.NET Web Forms, C#, SQL Server and a layered architecture that separates presentation logic, business rules, data access and database operations. The main technical challenge addressed by the project is not simply the creation of stock records, but the reliable control of stock movement when different users have different responsibilities and stock quantities must remain consistent across purchasing, selling, returning and adjustment workflows.

The completed system supports authentication, hierarchical role-based access control, access-request management, user administration, inventory item management, suppliers, customers, purchase orders, sales orders, returns, stock adjustments, low-stock email alerts, stock movement history and system audit logs. Stock quantities are not treated as ordinary editable fields. Instead, stock changes are routed through controlled operational workflows and database stored procedures, allowing the system to prevent negative stock, reserve quantities for draft sales orders, validate selling prices, record every stock-changing event and maintain traceability through a unified StockLedger table. The sales workflow includes additional business rules to prevent sales below the configured selling price and to prevent the same item being added to a sales order using inconsistent unit prices. Returns and stock adjustments also update inventory immediately while enforcing maximum permitted quantities.

A significant part of the implementation focuses on accountability. The AuditLogs module records user, role, event category, event type, page, device, browser, IP address, severity and timestamp. The audit module also supports filtering, autocomplete search, CSV export and Excel export. The Stock Movement History page provides a separate operational view of all inventory movements, including purchases, sales, sales returns, purchase returns and manual adjustments.

The evaluation demonstrates that the project satisfies the main requirements: role-based access restrictions are enforced, core stock workflows operate correctly, invalid quantities are rejected, out-of-stock products cannot be added to sales orders, sales approval prevents zero-total orders, returns are capped by original transaction quantities, purchase returns cannot exceed available stock, and audit evidence is generated for important actions. The project shows that even a conventional ASP.NET Web Forms and SQL Server stack can deliver a dependable stock-control system when access control, transactional integrity, validation, auditability and database-backed business rules are treated as first-class design concerns.

Acknowledgements

I would like to express my sincere thanks to my project supervisor for his guidance, feedback and support throughout the development of this project. His suggestions helped me to think beyond a basic inventory system and encouraged me to improve the system with stronger workflows, better validation, controlled access and more realistic stock movement handling. Several parts of the final system, including the approval-based stock changes, auditability and structured user permissions, became stronger because of this guidance.

I would also like to thank my principal marker for the feedback given during the interim stage. The comments helped me understand that the project needed to show a clearer relationship between the aims, requirements, design, implementation and evaluation. The suggestion to consider machine learning for future stock forecasting also helped me think about how the system could be extended beyond its current functionality. Although machine learning was not implemented in the final version, it has been included as a future development direction because the stock movement history created by this system could provide useful data for demand prediction.

Finally, I would like to thank the teaching staff and my peers for their support during the project. Their feedback helped me test the system from different user perspectives and improve the usability of the final application.

Table of Contents

Abstract.....	2
Acknowledgements	3
List of Figures	7
List of Tables.....	7
List of Listings.....	8
1. Introduction	9
1.1 Research Question.....	9
1.2 Aim.....	9
1.3 Objectives	10
1.4 Scope	10
1.5 project contribution	10
1.6 Report Structure	11
2. Background and Literature Review.....	11
2.1 Inventory Control and Stock Integrity	11
2.2 Role-Based Access Control	11
2.3 Database Transactions and ACID Properties	11
2.4 Concurrency and Overselling Risk	12
2.5 Secure Database Access	12
2.6 Audit Logging and Accountability.....	13
2.7 Layered Architecture and ASP.NET Web Forms	13
2.8 Evolution of the System Design	13
3. Requirements Analysis	13
3.1 Stakeholders	13
3.2 Functional Requirements	14
3.3 Non-Functional Requirements	15
3.4 Role-Based Access Matrix.....	16
3.5 Requirements Traceability	16
4. System Design.....	17
4.1 Design Overview.....	17
4.2 High-Level Architecture	18
4.3 Role-Based Design	18
4.4 Workflow Design	19
4.4.1 Purchase Approval Workflow	19
4.4.2 Sales Approval Workflow	20
4.4.3 Sales Return Workflow	20
4.4.4 Purchase Return Workflow	21
4.4.5 Stock Adjustment Workflow	23
4.5 Stock Movement Rules.....	23

4.6 User Interface Design	24
5. Database Design	24
5.1 Database Design Overview	24
5.2 Main Database Tables	25
5.3 InventoryItems Table	26
5.4 StockLedger Table	27
5.5 AuditLogs Table	28
5.6 Stored Procedures	29
6. Implementation	31
6.1 Technology Stack	31
6.2 Authentication and Session Validation	32
6.3 Access Requests and User Management	33
6.4 Inventory Management	35
6.5 Supplier and Customer Management	36
6.6 Purchase Order Implementation	37
6.7 Sales Order Implementation	40
6.8 Sales Approval and Low-Stock Email Alerts	44
6.9 Returns Implementation	45
6.10 Stock Adjustment Implementation	48
6.11 Stock Movement History Implementation	51
6.12 System Audit Logs Implementation	52
7. Testing and Evaluation	54
7.1 Testing Strategy	54
7.2 Functional Testing Results	54
7.3 Role-Based Access Testing	55
7.4 Validation and Negative-Stock Testing	56
7.5 Stock Movement Testing	57
7.6 Audit Log Evaluation	57
7.7 Security Testing	58
7.8 Performance and Scalability Observation	58
7.9 Usability Evaluation	59
7.10 Critical Evaluation of Testing	59
8. Project Management and Development Process	60
8.1 Development Approach	60
8.2 Project Phases	60
8.3 Risk Management	61
8.4 Changes During Development	62
8.5 Response to Review Feedback	62
9. Critical Evaluation	62
9.1 Strengths	62
9.2 Limitations	63

9.3 Evaluation Against Research Question.....	64
10. Technical Achievement	64
10.1 Technical Achievement Summary.....	64
10.2 Code Extract 1: Sales Item Validation and Reservation	65
10.3 Code Extract 2: Return Quantity Validation	66
10.4 Code Extract 3: Purchase Approval Stored Procedure	66
10.5 Code Extract 4: Audit Logging with Request Context.....	70
10.7 Autocomplete with hidden ID validation.....	72
11. Conclusion and Future Work	74
11.1 Conclusion	74
11.2 Future Work.....	74
References	75
Appendices	76
Appendix A: Screenshot Checklist	76
Appendix B – Key Code Listings	77
Appendix B.1 – Sales Quantity Validation and Draft Reservation	77
Appendix B.2 – Return Quantity Validation	77
Appendix B.3 – Purchase Approval Stored Procedure	77
Appendix B.4 – Audit Logging with Request Context.....	77
Appendix B.5 – Low-Stock Email Alert Logic	77
Appendix C: Running the Application.....	77

List of Figures

Figure 4.1 – Layered Architecture Diagram
Figure 4.2 – Purchase Workflow
Figure 4.3 – Sales Workflow
Figure 4.4 – Sales Return Workflow
Figure 4.5 – Purchase Return Workflow
Figure 4.6 – Stock Adjustment Workflow
Figure 5.1 – Database ER Diagram
Figure 6.1 – Login Page
Figure 6.2 – Dashboard Page
Figure 6.3 – User Management Page
Figure 6.4 – Access Request Page
Figure 6.5 – Inventory Management Page
Figure 6.6 – Supplier Management Page
Figure 6.7 – Customer Management Page
Figure 6.8 – Purchase Order List Showing Draft and Approved Records
Figure 6.9 – Purchase Items Page with Item List, Order Total and Approval Action
Figure 6.10 – Sales Order List
Figure 6.11 – Sales Order Creation Modal
Figure 6.12 – Sales Items Page with Draft Order Details
Figure 6.13 – Sales Validation: Price Below Selling Price
Figure 6.14 – Sales Validation: Quantity Exceeds Available Stock
Figure 6.15 – Sales Validation: Duplicate Item with Different Price
Figure 6.16 – Sales Validation: Out-of-Stock Item Disabled
Figure 6.17 – Low-Stock Email Alert Sent After Sales Approval
Figure 6.18 – Sales Return Creation Form with Calculated Maximum Return Quantity
Figure 6.19 – Sales Return Validation When Quantity Exceeds the Allowed Limit
Figure 6.20 – Purchase Return Form with Maximum Return Quantity Based on Purchased and Available Stock
Figure 6.21 – Stock Adjustment Page
Figure 6.22 – Adjustment OUT Validation Preventing Negative Stock
Figure 6.23 – Stock Movement History Page Showing Filtered Stock Transactions
Figure 6.24 – System Audit Logs Page with Category Filters and Export Options
Figure 6.25 – Audit Log Excel Export Evidence

List of Tables

Table 2.1 – ACID Properties and Relevance to the Project
Table 2.2 – Critical Comparison with Existing Approaches
Table 3.1 – Stakeholders and Responsibilities
Table 3.2 – Functional Requirements
Table 3.3 – Non-Functional Requirements
Table 3.4 – Role-Based Access Matrix
Table 3.5 – Requirements Traceability Matrix
Table 4.1 – Stock Movement Rules
Table 5.1 – Main Database Tables
Table 5.2 – Key Fields in the InventoryItems Table
Table 5.3 – StockLedger Transaction Types
Table 5.4 – Key Fields in the AuditLogs Table
Table 5.5 – Example Logged System Events
Table 5.6 – Important Stored Procedures

Table 6.1 – Technology Stack

Table 7.1 – Functional Testing Results

Table 7.2 – Role-Based Access Testing

Table 7.3 – Validation and Negative-Stock Testing Results

Table 7.4 – Stock Movement Testing

Table 7.5 – Audit Log Evaluation

Table 7.6 – Security Testing

Table 7.7 – Usability Strengths Identified During Testing

Table 7.8 – Usability Limitations and Improvements

Table 8.1 – Project Phases

Table 8.2 – Risk Management

Table 9.1 – Evaluation Against the Research Question

Table 10.1 – Technical Achievement Summary

List of Listings

Listing 10.1 – Sales Quantity Validation and Draft Reservation Logic

Listing 10.2 – Return Quantity Validation Before Stock Movement

Listing 10.3 – Purchase Approval Stored Procedure with Transaction Handling and Stock Ledger Recording

Listing 10.4 – Audit Logging with Page, Device, Browser and Severity Context

Listing 10.5 – Low-Stock Email Alert Generation

1. Introduction

Inventory management is often understood as the process of recording items and quantities. In practice, however, a stock-control system becomes more complex when several users interact with the same data and when stock can change for different business reasons. A warehouse user may create a sales order, an administrator may approve it, a supplier may deliver stock, a customer may return goods, and an auditor may later need to check who performed each action. If the system only stores the current quantity of each item, it becomes difficult to explain how that quantity was reached or whether every movement was valid.

This project develops a secure web-based Stock Control Management System for a warehouse-style environment. The system was implemented using ASP.NET Web Forms, C#, SQL Server and stored procedures. The application is designed around controlled stock movement rather than unrestricted quantity editing. This means that stock changes are not made casually by editing a number in the inventory table. Instead, stock is changed through business workflows such as purchase approval, sale approval, sales return, purchase return and stock adjustment.

The project was built for an enterprise warehouse context rather than a direct retail shop. This distinction affected the design of the sales workflow. In a shop, a sale may immediately reduce stock at the point of payment. In this project, a Staff user can create a draft sales order and add requested items for a customer, but the sale must be approved before final stock movement is recorded. This reflects a business environment where order preparation and authorisation are separate responsibilities.

The system also implements role-based access control. Four roles are used: SuperAdmin, Admin, Staff and Auditor. Each role has different access rights and interface visibility. SuperAdmin has the widest access. Admin users can manage operational workflows and approve transactions. Staff users can create day-to-day operational records but cannot perform sensitive approval actions. Auditors can view records and audit evidence but cannot create, update or delete operational data. This separation of duties is important because stock control involves sensitive actions such as approving sales, deleting records and manually adjusting inventory.

Another important part of the project is traceability. The system records stock movements in a StockLedger table and records user/system events in an AuditLogs table. The Stock Movement History page shows all inventory movements, including purchases, sales, returns and adjustments. The System Audit Logs page records user identity, role, event category, event type, description, page name, device, browser, IP address, severity and timestamp. This gives the system accountability: when stock changes, the system can show what changed, why it changed, who caused the change and when it occurred.

1.1 Research Question

The project is guided by the following research question:

How can a secure role-based inventory management system be designed to maintain transactional integrity, stock consistency and audit traceability in a multi-user web environment?

This question connects the practical software artefact to wider software engineering concerns such as access control, database integrity, business-rule validation, transaction design, usability and auditability.

1.2 Aim

The aim of this project is to design, implement and evaluate a secure web-based stock control management system that enforces hierarchical role-based access control while maintaining transactional integrity, stock consistency and audit traceability across inventory workflows

1.3 Objectives

The objectives of the project are:

- To implement secure authentication and session-based access control.
- To define and enforce a hierarchical role model using SuperAdmin, Admin, Staff and Auditor roles.
- To design a layered architecture separating ASPX pages, code-behind logic, service classes, data access classes and SQL Server stored procedures.
- To implement access-request and user-management functionality.
- To implement inventory item management with support for SKU, quantity, price, selling price and minimum stock level.
- To implement supplier and customer management with search, validation and role-based visibility.
- To implement purchase orders and purchase items, with stock increased only after approval.
- To implement sales orders and sales items, with available-stock validation, selling-price validation, same-item price consistency and approval-controlled stock movement.
- To implement a reservation-style sales workflow where draft sales items reduce the available quantity shown for that order.
- To implement sales returns and purchase returns using cascading autocomplete and maximum return quantity validation.
- To implement stock adjustments for controlled manual corrections.
- To prevent negative stock across sales, returns and adjustment workflows.
- To implement stock movement history for purchases, sales, returns and adjustments.
- To implement audit logging for page access, operational actions, system errors and unauthorised access attempts.
- To implement low-stock email alerts for Admin and SuperAdmin users.
- To evaluate the system using functional testing, role testing, validation testing, stock movement testing and audit evidence.
- To critically evaluate the strengths, limitations and possible future development of the system.

1.4 Scope

The project covers the main features required for a controlled warehouse stock-management application. These include authentication, access requests, user management, inventory management, suppliers, customers, purchase orders, sales orders, sales items, returns, stock adjustments, stock movement history, low-stock email alerts and audit logs.

The project does not attempt to become a full enterprise resource planning system. It does not include payment processing, invoicing, financial accounting, stock valuation, barcode scanner hardware integration, delivery management or customer relationship management. These areas are outside the scope of the project and are discussed later as possible future work.

The scope is intentionally focused on stock integrity, role-based control and auditability. The main contribution is not the number of pages alone, but the way those pages work together to protect stock data and provide traceability.

1.5 project contribution

The main contribution of this project is not only the creation of a stock control application, but the development of a controlled workflow for warehouse stock movement. The system separates routine users from approval users, records the reason behind stock changes, prevents invalid stock operations, and maintains traceability through audit logs and stock movement history. During development, the project evolved from a simple inventory CRUD system into a workflow-based stock management system with purchasing, sales, returns, adjustments, role-based permissions, autocomplete validation, low-stock email alerts, and exportable audit evidence.

1.6 Report Structure

Chapter 2 discusses background concepts including role-based access control, database transactions, stock integrity, secure database programming and audit logging. Chapter 3 defines the requirements and explains how they relate to the implemented system. Chapter 4 describes the system design, including architecture, role access, workflows and user interface design. Chapter 5 explains the database design. Chapter 6 describes the implementation of each major module. Chapter 7 presents testing and evaluation. Chapter 8 discusses project management and development process. Chapter 9 critically evaluates the system. Chapter 10 highlights the main technical achievement. Chapter 11 concludes the report and proposes future work.

2. Background and Literature Review

2.1 Inventory Control and Stock Integrity

Inventory control is concerned with keeping stock records accurate while supporting day-to-day operations such as purchasing, selling, returning and adjusting items. In a simple system, stock quantity may only be stored as one editable number. However, this can create problems because it does not explain **why** the quantity changed.

For this project, stock integrity means that inventory quantities should only change through controlled business workflows. A purchase approval should increase stock, a sales approval should decrease stock, a sales return should increase stock, and a purchase return or adjustment OUT should reduce stock. This approach is more reliable than allowing users to freely edit item quantities without a reason or record.

The main design idea used in this project is therefore **stock movement traceability**. The current quantity of an item is still important, but the system also records the movement that caused the change. This makes the system easier to review and audit when stock values appear incorrect.

2.2 Role-Based Access Control

Role-Based Access Control is a common method of managing permissions in multi-user systems [1]. Instead of assigning permissions individually to every user, permissions are associated with roles. Users are then assigned to roles according to their responsibilities.

This project uses four roles:

Role	Responsibility
SuperAdmin	Full system control, including users, access requests and all operational modules.
Admin	Operational management, approvals, inventory control and audit access.
Staff	Day-to-day operational work such as creating sales and purchase records, but without sensitive approval rights.
Auditor	Read-only access to operational evidence, stock movement history and audit logs.

Table 2.1 - Role-Based Access Control

This role model supports the security aim of the project by limiting sensitive actions such as approvals, stock adjustments, user management and audit access.

2.3 Database Transactions and ACID Properties

Stock-changing operations often require several database changes to happen together. For example, approving a purchase may need to update the purchase status, increase item quantity, insert a stock ledger record and record an audit event. If only part of that process succeeds, the database can become inconsistent.

Database transactions are therefore important because they allow related database operations to complete as one unit of work. The ACID properties of transactions are relevant to this project because stock operations should be reliable, consistent and recoverable if an error occurs [2], [10].

Property	Meaning in this project
Atomicity	A stock-changing operation should complete fully or fail safely.
Consistency	Stock rules such as “quantity cannot become negative” should be preserved.
Isolation	Operations should not interfere with each other incorrectly.
Durability	Once a valid stock movement is saved, it should remain recorded.

Table 2.2 – ACID Properties

The project uses service workflow classes and SQL Server stored procedures to support these ideas. Important stockchanging logic is centralised so that validation, stock updates and movement records are not spread randomly across different pages.

2.4 Concurrency and Overselling Risk

Concurrency occurs when more than one user interacts with the same data at the same time. In a stock-control system, this can create overselling or lost-update problems. For example, two users may try to sell the last available units of an item.

This project reduces the risk through validation in the sales workflow. The sales item page checks available stock before adding an item and also considers the quantity already added to the same draft sales order. Final stock deduction is then controlled through the approval workflow.

The project does not include full concurrent-user stress testing, so this remains a limitation. However, the design does include stock validation and approval-controlled movement to reduce the chance of invalid stock changes.

2.5 Secure Database Access

The system uses ADO.NET with SQL Server stored procedures. User input is passed through parameters rather than being directly concatenated into SQL strings. This helps reduce SQL injection risk and makes database operations clearer [5], [10].

For example, data access classes call stored procedures such as `sp_Customer_Insert`, `sp_Sales_Create`, `sp_HandleReturn`, `sp_StockAdjustment`, `sp_GetStockLedgerFiltered` and `sp_AuditLogged_Insert`. These procedures separate database operations from the ASPX page logic, which improves maintainability and reduces repeated SQL code across the application.

The use of stored procedures also supports the layered architecture of the project. The Web Forms pages handle user interaction, service classes apply business rules, DAL classes call database operations, and SQL Server stored procedures handle the final data changes. This makes the system easier to understand and test because the database access logic is not scattered throughout the user interface code.

2.6 Audit Logging and Accountability

Audit logging is an important part of this project because a stock-control system should be able to explain user actions. If a sale is approved, a purchase is deleted, a return is created, or an unauthorised user attempts to access a restricted page, the system should record that event.

Authentication and authorisation guidance supports the need to record and review security-relevant events in a controlled system [6], [7]. In this project, the audit log records the user, role, event category, event type, description, page name, device, browser, IP address, severity and timestamp.

This supports accountability because important actions can be reviewed after they happen. It also supports the Auditor role, because auditors need access to evidence without being allowed to change operational data. This separation is important in the system because administrators can manage and approve stock-related work, while auditors can review activity independently.

2.7 Layered Architecture and ASP.NET Web Forms

ASP.NET Web Forms supports event-driven web application development, while ADO.NET provides data access functionality for communicating with SQL Server [4], [6]. However, Web Forms applications can become difficult to maintain if every page contains business logic, validation and database code together.

To reduce this problem, the project uses a layered structure. The ASPX page handles the interface, the code-behind handles events, service classes apply business rules, DAL classes call the database, and stored procedures perform the database operations.

This separation made the project easier to extend. For example, sales validation, return handling and audit logging could be implemented as workflow logic rather than being scattered across unrelated pages.

2.8 Evolution of the System Design

At the start of the project, the system was planned mainly as an inventory management application for storing items, suppliers, customers and stock quantities. During implementation, it became clear that this would not be enough for a realistic stock-control system.

The design changed to focus more on **stock movement** instead of only **stock quantity**. This was an important shift. A stock value can change for many reasons, including purchase approval, sales approval, sales return, purchase return, manual adjustment or correction of an error. If the system only stores the latest quantity, it becomes difficult to explain how that quantity was reached.

Because of this, the final design records movement type, reference number, party name, user, date and reason. This made the system more complex, but it also improved traceability and made the project stronger than a simple CRUD inventory application.

3. Requirements Analysis

3.1 Stakeholders

The system is designed around four main user roles. These roles represent the users who interact with the application and the responsibilities they have within the stock-control workflow.

SuperAdmin	Needs full control over the system, including user management, access requests, inventory data, approval workflows, stock movement history and audit logs. This role has the highest level of access.
Admin	Needs to manage daily inventory operations, approve purchase and sales transactions, manage most user-related tasks and review system activity. Admin has similar access to SuperAdmin but cannot delete users.
Staff	Needs to create day-to-day operational records such as purchase orders, sales orders and returns. Staff should not approve final stock-changing transactions or access restricted administration features.
Auditor	Needs read-only access to operational records, stock movement history and audit logs. Auditor should be able to review evidence but not change stock, users or transaction data.

Table 3.1 – Stake Holders and responsibilities

The role model was kept focused to avoid unnecessary complexity. Earlier versions considered broader business roles, but the final implementation uses only the roles that exist in the system: SuperAdmin, Admin, Staff and Auditor.

3.2 Functional Requirements

The functional requirements describe the main behaviours that the system must support.

ID	Requirement	Implemented Feature	Status
FR1	Users must authenticate before accessing the system.	Login, session validation and timeout handling.	Met
FR2	The system must support role-based access control.	SuperAdmin, Admin, Staff and Auditor roles.	Met
FR3	Users must be able to request system access.	Access request workflow with admin approval/rejection.	Met
FR4	Admin users must be able to manage users.	User management page with role/status editing.	Met
FR5	Inventory items must be created, updated and viewed.	Inventory module with item CRUD, validation and search.	Met
FR6	Suppliers must be managed.	Supplier management module.	Met
FR7	Customers must be managed.	Customer management module.	Met
FR8	Purchase orders must be created.	Purchase order workflow.	Met
FR9	Purchase items must be added to purchase orders.	Purchase item page with item search, quantity and price.	Met
FR10	Purchase approval must increase stock.	Purchase approval workflow updates inventory quantity.	Met
FR11	Sales orders must be created.	Sales order workflow.	Met
FR12	Sales items must be added to sales orders.	Sales item page with item search, quantity and price.	Met
FR13	Sales must not allow unavailable stock to be sold.	Available stock and reserved quantity validation.	Met
FR14	Sales must not allow selling below configured selling price.	Selling-price validation.	Met
FR15	The same item must not be added to one sales order with different prices.	Duplicate item price rule.	Met
FR16	Sales approval must decrease stock.	Sales approval workflow updates inventory quantity.	Met
FR17	Empty or zero-total sales must not be approved.	Approval validation before sale completion.	Met
FR18	Sales returns must increase stock.	Sales return workflow.	Met
FR19	Purchase returns must decrease stock.	Purchase return workflow.	Met

FR20	Returns must not exceed the remaining valid quantity.	Remaining quantity validation using original transaction and previous returns.	Met
FR21	Stock adjustments must support IN and OUT movements.	Stock adjustment module.	Met
FR22	Adjustment OUT must not create negative stock.	Stock validation before adjustment.	Met
FR23	All stock-changing events must be traceable.	Stock Movement History page.	Met
FR24	Important user and system actions must be logged.	System Audit Logs module.	Met
FR25	Audit logs must be searchable and exportable.	Search, filters, CSV export and Excel export.	Met
FR26	Low-stock alerts must be sent when stock falls below minimum level.	Email alert to Admin and SuperAdmin users.	Met

Table 3.2 – Functional Requirements

The functional requirements focus on controlled stock movement rather than simple inventory editing. This matches the project aim because stock accuracy depends on validated workflows, not just item records.

3.3 Non-Functional Requirements

The non-functional requirements define the quality expectations of the system.

ID	Requirement	System Response	Evaluation Approach
NFR1	Security	Authentication, session validation, role checks and parameterised SQL commands.	Tested by attempting unauthorised page access and invalid login attempts.
NFR2	Integrity	Stored procedures, validation rules and controlled stock workflows.	Tested using negative-stock, over-selling and overreturning scenarios.
NFR3	Traceability	Stock Movement History and AuditLogs records.	Checked that purchases, sales, returns, adjustments and user actions created reviewable records.
NFR4	Maintainability	Layered architecture using ASPX/code-behind, Services, DAL classes and stored procedures.	Evaluated through separation of UI, business rules and database access.
NFR5	Usability	Consistent layout, modal forms, autocomplete, filters and clear validation messages.	Evaluated through manual walkthroughs of common workflows.
NFR6	Reliability	Negative-stock prevention, approval-controlled workflows and validation before stock updates.	Tested by attempting invalid sales, returns and adjustments.
NFR7	Accountability	Audit logging with user, role, page, browser, IP address, severity and timestamp.	Verified through the System Audit Logs page and export evidence.
NFR8	Scalability awareness	Search procedures, filters and autocomplete to reduce unnecessary browsing through large tables.	Evaluated by checking that major list pages support searching/filtering rather than only full-table browsing.
NFR9	Data quality	Validators for required fields, quantity, price, email format and phone numbers.	Tested through invalid form submissions and edge-case inputs.

Table 3.3 - Non-functional requirements and evaluation approach.

These requirements shaped the implementation. For example, traceability led to the Stock Movement History and Audit Logs pages, while integrity led to validation in sales, returns and adjustments.

3.4 Role-Based Access Matrix

The role-based access matrix defines which roles can access key system features.

Feature	SuperAdmin	Admin	Staff	Auditor
Dashboard access	Yes	Yes	Yes	Yes
Inventory view	Yes	Yes	Yes	Yes
Inventory create/update	Yes	Yes	No	No
Inventory delete	Yes	No	No	No
Supplier view	Yes	Yes	Yes	Yes
Supplier create/update	Yes	Yes	Limited	No
Supplier delete	Yes	No	No	No
Customer view	Yes	Yes	Yes	Yes
Customer create/update	Yes	Yes	Limited	No
Customer delete	Yes	No	No	No
Purchase order create	Yes	Yes	Yes	No
Purchase item add/delete	Yes	Yes	Yes	No
Purchase approval	Yes	Yes	No	No
Sales order create	Yes	Yes	Yes	No
Sales item add/delete	Yes	Yes	Yes	No
Sales approval	Yes	Yes	No	No
Returns create	Yes	Yes	Yes	No
Stock adjustment create	Yes	Yes	No	No
Stock adjustment view	Yes	Yes	No	Yes
Stock Movement History	Yes	Yes	No	Yes
User management	Yes	Limited	No	No
Access requests	Yes	Yes	No	No
System Audit Logs	Yes	Yes	No	Yes
CSV/Excel export of logs	Yes	Yes	No	Yes

Table 3.4 – Role-Based Access Matrix.

The access design separates everyday data entry from approval and audit responsibilities. This is important because Staff users can support daily operations, while Admin and SuperAdmin users control higher-risk actions such as approvals and adjustments. Auditors are given evidence access without permission to change operational data..

3.5 Requirements Traceability

The traceability matrix links the project requirements to the implemented modules and testing evidence.

Objective	Related Requirements	Implemented Module	Evaluation Evidence
Secure authentication	FR1, NFR1	Login page, session handling and BasePage validation.	Login testing, invalid-login testing, session timeout behaviour and protected page access tests.
Hierarchical rolebased access control	FR2, NFR1	RequireRole() checks, master-page menu visibility and role-specific page actions.	Role-based screenshots, access matrix and restricted page testing.
Inventory management	FR5	Inventory module with CRUD, validation, search and bulk upload.	Inventory screenshots, add/update tests, search tests and validation tests.
Supplier and customer management	FR6, FR7	Supplier and customer modules with search, validation and role-based actions.	Add/update/search screenshots and phone/email validation tests.
Purchase control	FR8–FR10	Purchase order and purchase item workflow.	Purchase creation, purchase item entry, approval and stock IN movement tests.
Sales control	FR11–FR17	Sales order and sales item workflow.	Stock reservation, selling-price validation, duplicate-price validation, out-of-stock handling and approval tests.
Return control	FR18–FR20	Sales return and purchase return workflow.	Cascading autocomplete screenshots, remaining quantity validation and stock movement tests.
Manual stock correction	FR21–FR22	Stock adjustment module.	Adjustment IN/OUT tests and adjustment OUT negative-stock rejection test.
Traceability	FR23–FR25, NFR3, NFR7	Stock Movement History and System Audit Logs.	Movement history screenshots, audit log screenshots, category filters and CSV/Excel export evidence.
Low-stock monitoring	FR26	Low-stock email service.	Low-stock alert email screenshot and sales approval test that triggers the alert.

Table 3.5 – Requirements Traceability Matrix

This matrix shows that the system was not tested only at page level. The main requirements were linked to specific workflows and evidence, which makes the evaluation more structured.

4. System Design

4.1 Design Overview

The system design is based on the idea that inventory quantity should not be treated as a casual editable field. Users can create records such as purchases, sales, returns and adjustments, but the actual stock movement is controlled through validation, approval and logging.

The main design decisions were:

1. Use role-based access control to separate responsibilities.
2. Use layered architecture to improve maintainability.
3. Use stored procedures for database operations.
4. Prevent uncontrolled direct stock quantity changes.
5. Record stock movement history for stock-changing workflows.
6. Record audit logs for important user and system events.
7. Use autocomplete and hidden IDs to reduce invalid selections.
8. Use approval workflows for purchase and sales stock movement.

These decisions made the system closer to a real stock-control workflow because ordinary data entry and high-risk stock movement are separated.

4.2 High-Level Architecture

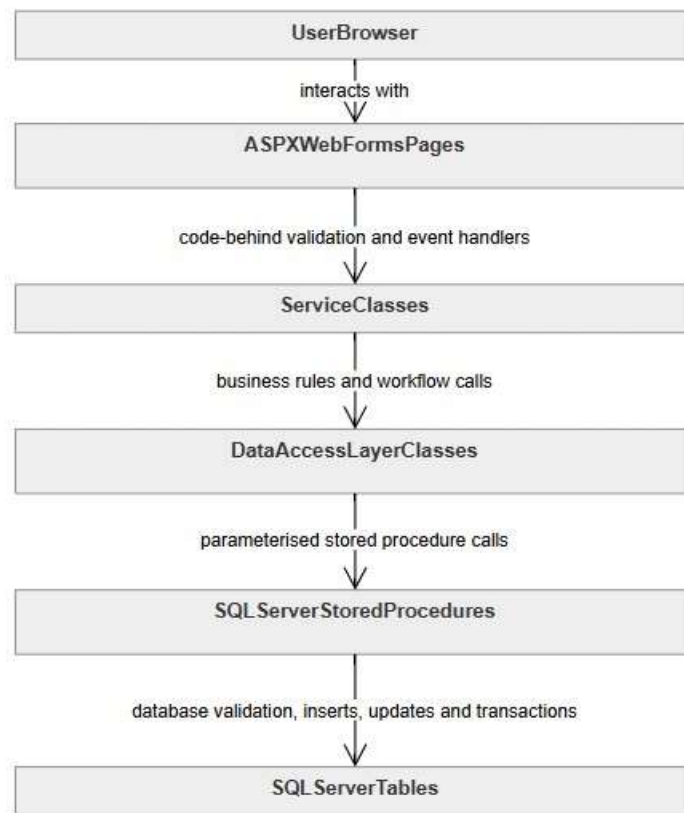


Figure 4.1 – Layered Architecture of the stock control system

The user interacts with the ASP.NET Web Forms pages through the browser. The ASPX pages and code-behind files handle user events, validation controls and page-level behaviour. The service classes then apply the main business rules, such as checking whether a sale can be approved, whether a return quantity is valid or whether a stock adjustment would create negative stock. The data access layer is responsible for calling SQL Server stored procedures using parameterised commands. The stored procedures then perform the final database operations against the SQL Server tables.

This design was used consistently across the project. For example, the Customer page uses CustomerService, which calls CustomerDAL, which then executes stored procedures such as sp_Customer_Insert and sp_Customer_GetAll. Similarly, the Sales Items page uses SalesItemService, which calls SalesItemDAL and executes stored procedures for adding sales items, checking available stock and approving sales.

This structure helped keep the project maintainable. The page code mainly deals with user interaction, while the service layer deals with business decisions and the DAL deals with database communication. This separation was especially useful when implementing stock movement rules because the same pattern could be reused across purchases, sales, returns and adjustments.

4.3 Role-Based Design

Role-based access control is applied at two levels.

The first level is **backend page protection**. Protected pages check the logged-in user's role before allowing access. This is important because hiding a button or sidebar link is not enough. A user could still attempt to access a restricted page directly through the URL.

The second level is **interface visibility**. Pages hide or disable actions that are not appropriate for the current role. For example, Staff users can create operational records but cannot approve sales. Auditors can view stock movement and audit evidence, but they cannot create, edit or approve operational records.

This two-level approach makes the system safer and easier to use. Users only see actions relevant to their responsibility, while backend checks still protect the system if someone tries to bypass the interface.

4.4 Workflow Design

The system is designed around controlled stock workflows rather than direct stock editing. This was an important design decision because inventory quantity should not change without a clear business reason. In this system, stock can change through purchase approval, sales approval, returns or stock adjustments. Each movement is recorded so that the system can explain why stock increased or decreased.

4.4.1 Purchase Approval Workflow

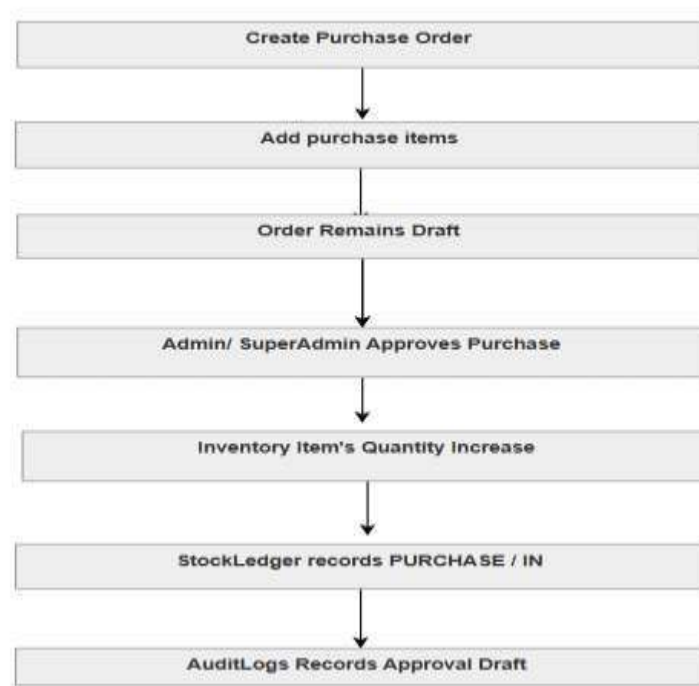


Figure 4.2 – Purchase approval Workflow

In the purchase workflow, a user creates a purchase order and adds purchase items. The order remains in **Draft** status until an authorised Admin or SuperAdmin approves it.

Stock is not increased when the draft purchase is created. Stock increases only after approval. At that point, the system updates inventory quantity, records a **PURCHASE / IN** movement in StockLedger, and records the approval activity in AuditLogs.

This design prevents incomplete or unapproved purchase orders from affecting real inventory quantity.

4.4.2 Sales Approval Workflow

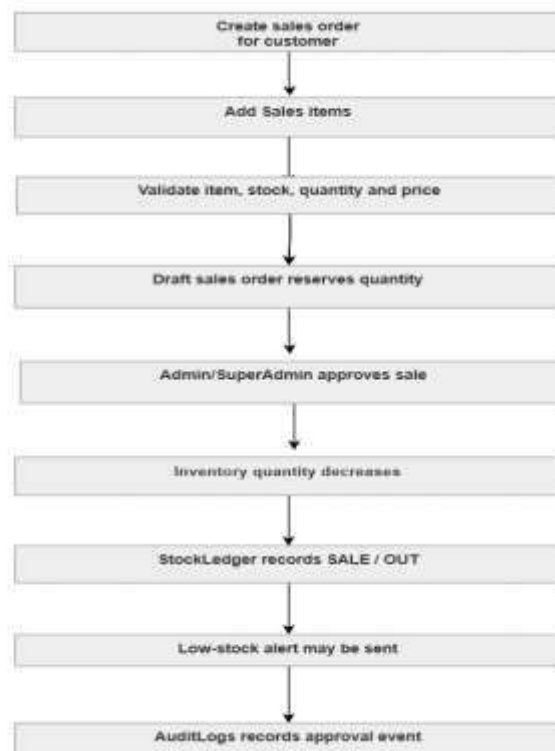


Figure 4.3 – Sales approval Workflow

In the sales workflow, a user creates a sales order for a customer and adds sales items while the order is still in **Draft** status. The system validates item selection, available stock, quantity and selling price before allowing items to be added.

Final stock deduction happens only when an authorised Admin or SuperAdmin approves the sale. Approval decreases inventory quantity, records a **SALE / OUT** movement in StockLedger, checks for low-stock alerts and records the approval in AuditLogs.

This design reduces overselling risk because stock is checked before items are added and final movement is controlled through approval.

4.4.3 Sales Return Workflow

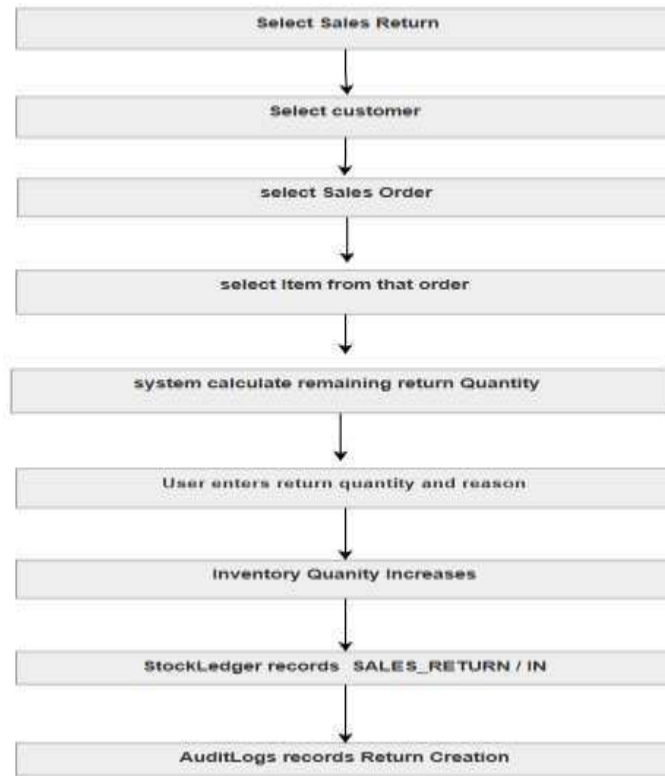


Figure 4.4 – Sales Return Workflow

The sales return workflow allows stock to be increased when a customer returns an item. The user selects the return type, customer, sales order and item. The system then calculates how much quantity can still be returned.

A sales return is only allowed if the entered quantity does not exceed the remaining valid return quantity. When the return is created, inventory quantity increases, a **SALES_RETURN / IN** movement is recorded in StockLedger, and the action is recorded in AuditLogs.

This prevents users from returning more stock than was originally sold.

4.4.4 Purchase Return Workflow

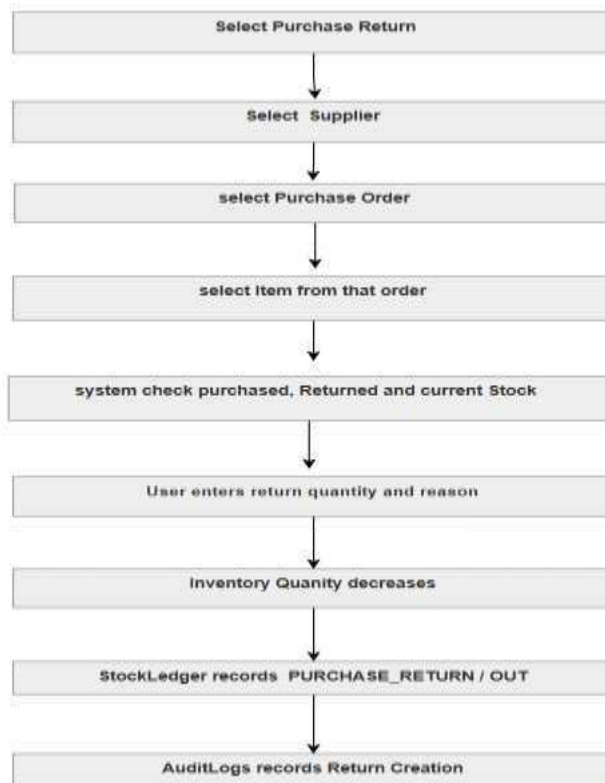


Figure 4.5 – Purchase Return Workflow

The purchase return workflow is used when stock is returned to a supplier. The user selects the supplier, purchase order and item, then enters the return quantity and reason.

Because a purchase return reduces inventory, the system checks that the return quantity is valid and that enough current stock exists. If the return is valid, inventory quantity decreases, a **PURCHASE_RETURN / OUT** movement is recorded in StockLedger, and the action is logged in AuditLogs.

This protects stock integrity because the system does not allow purchase returns to remove stock that is no longer available.

4.4.5 Stock Adjustment Workflow

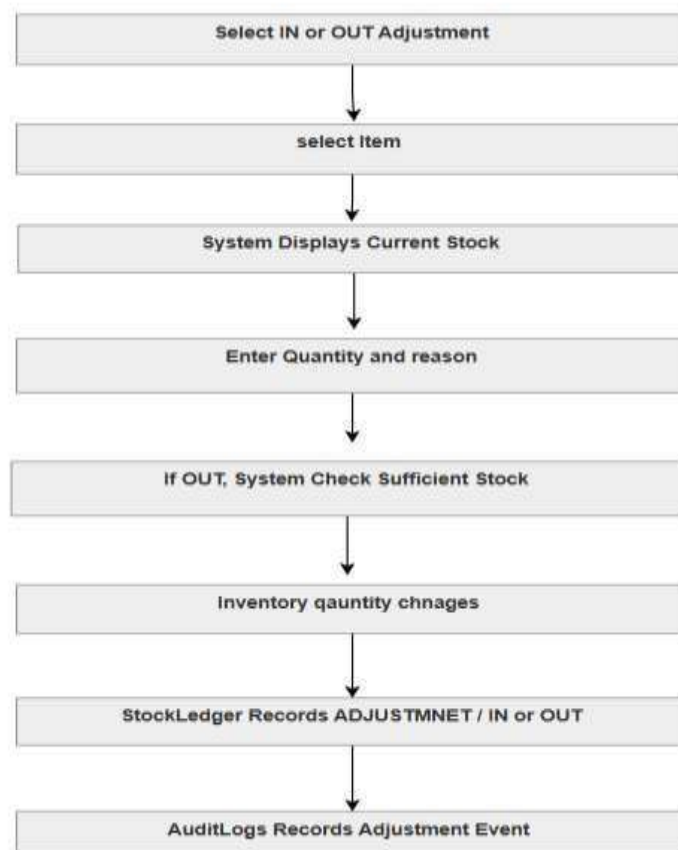


Figure 4.6 –Stock Adjustment Workflow

The stock adjustment workflow supports manual stock correction. This is needed when physical stock differs from recorded stock because of damage, loss, counting errors or correction of earlier mistakes.

The workflow supports two movement types:

Adjustment IN increases stock.

Adjustment OUT decreases stock.

For adjustment OUT, the system validates that the entered quantity does not exceed current stock. If valid, the adjustment updates inventory, records an **ADJUSTMENT / IN** or **ADJUSTMENT / OUT** movement in StockLedger, and records the event in AuditLogs.

This design allows manual correction while still preventing negative stock and preserving traceability.

4.5 Stock Movement Rules

The main stock movement rules are summarised below.

Workflow	Movement Type	Stock Effect	Main Validation
Purchase approval	IN	Increases inventory	Purchase order must contain valid purchase items before approval.
Sales approval	OUT	Decreases inventory	Sale must have a non-zero total and enough available stock.

Sales return	IN	Increases inventory	Return quantity cannot exceed sold quantity minus already returned quantity.
Purchase return	OUT	Decreases inventory	Return quantity cannot exceed purchased quantity minus already returned quantity, and must not exceed current stock.
Adjustment IN	IN	Increases inventory	Quantity must be greater than zero and a valid item must be selected.
Adjustment OUT	OUT	Decreases inventory	Quantity must be greater than zero and must not exceed current stock.

Table 4.1 – Stock movement rules

These rules show that every stock-changing workflow has a defined movement direction and validation rule. This keeps inventory changes consistent across the system.

4.6 User Interface Design

The user interface was designed to stay consistent across the main modules. Most pages follow the same structure: sidebar navigation, page title, search box, primary action button, modal form, GridView table, filters where needed and role-based action visibility.

This consistency reduces user confusion because customers, suppliers, sales, returns, adjustments, stock movement history and audit logs all follow a similar layout. Autocomplete is used where a user must select an existing record, such as a customer, supplier, order or item. In these cases, the system stores the selected database ID rather than trusting only the visible text.

Filter tabs are used on pages with larger or mixed data sets, such as returns, stock adjustments, stock movement history and system audit logs. This makes review pages easier to use because users can focus on one category at a time.

5. Database Design

5.1 Database Design Overview

The database was designed to support two main project goals: **stock integrity** and **traceability**. In a basic inventory system, it would be possible to store only the current quantity of each item. However, this would not explain why the quantity changed, who changed it, or which business process caused the change. For this reason, the database design separates item master data from stock movement records.

The InventoryItems table stores the current state of each stock item, including fields such as SKU, item name, category, unit price, selling price, current quantity, minimum stock level and low-stock alert status. This table answers the question: **“What is the current stock position?”**

However, the system also needs to answer a second question: **“How did the stock reach this position?”** This is handled through operational and movement-related tables, including PurchaseOrders, PurchaseOrderItems, SalesOrders, SalesOrderItems, Returns, StockAdjustments and StockLedger. These tables record the business reason for each stock change, such as a purchase approval, sale approval, sales return, purchase return or manual adjustment.

The most important database design decision was therefore to treat stock as a sequence of controlled movements rather than only a number stored in one table. This supports the main aim of the project because stock quantities are not updated without context. Each important stock-changing action can be linked to a reference number, movement type, party name, quantity, user and date.

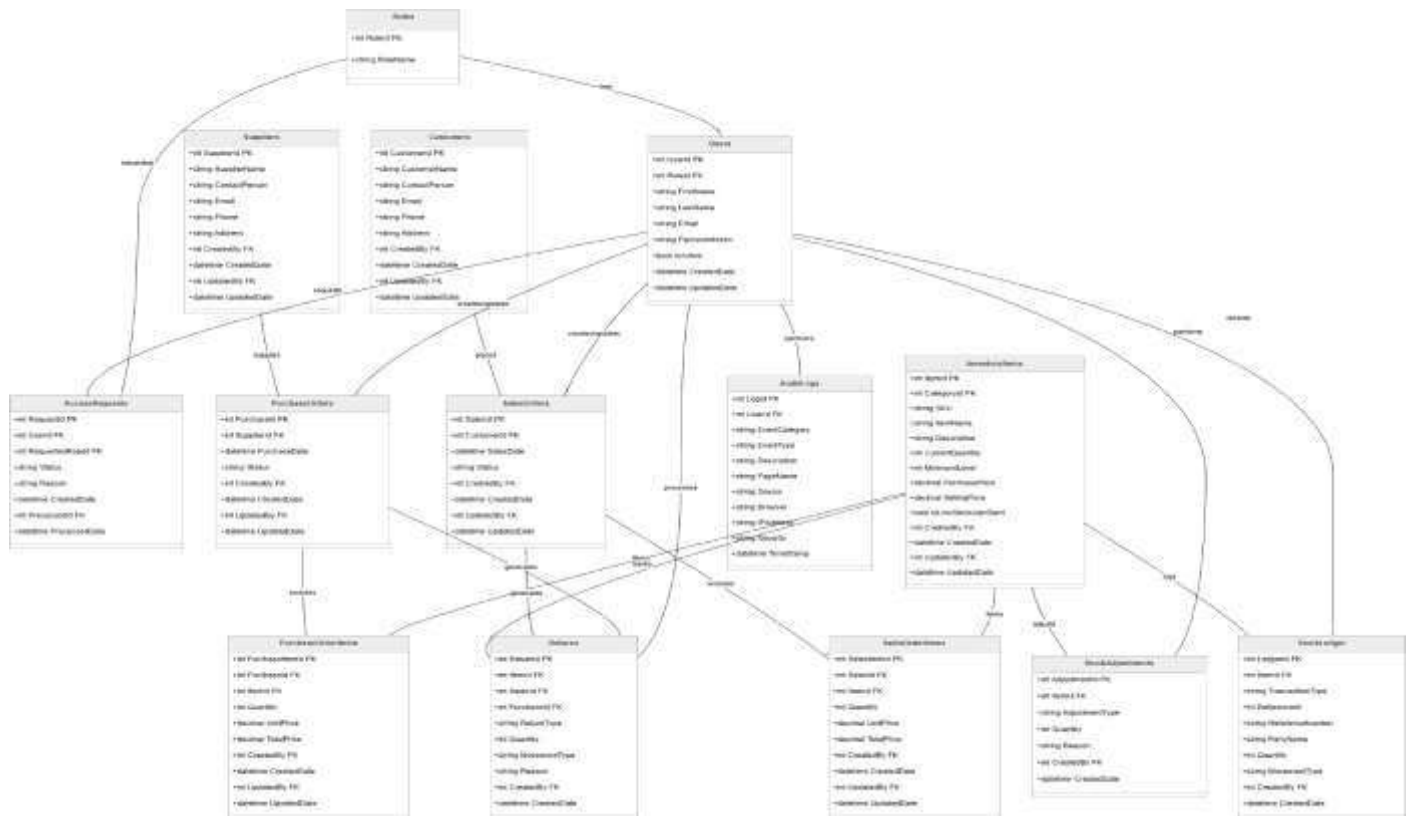


Figure 5.1 – Database ER Diagram

Figure 5.1 shows the main database entities used by the Stock Control System and how they support the operational workflows. The diagram separates master data, such as users, roles, suppliers, customers, categories and inventory items, from transactional data, such as purchase orders, sales orders, returns, stock adjustments, stock ledger records and audit logs.

The relationship between Roles and Users supports role-based access control, while the relationships between users and operational tables allow the system to record who created, updated or approved records. Supplier and customer tables are connected to purchase and sales workflows respectively. Purchase and sales order item tables connect business documents to individual inventory items. The Returns table supports both sales returns and purchase returns by linking returned stock to the original sale or purchase where applicable.

The StockLedger table acts as a central stock movement history. Instead of checking each module separately, the system can show all stock movements in one place using transaction type and movement type filters. This is important for auditability because a stock movement can be reviewed together with its reference number, item, party, quantity, movement direction and date.

The AuditLogs table has a different purpose from the stock ledger. While the stock ledger records stock movements, the audit log records user and system activity, such as page access, searches, approvals, errors and unauthorised access attempts. This distinction is important because it separates **stock traceability** from **user accountability**.

Overall, the database structure supports the project's workflow-based design. It allows the system to maintain current stock values while also preserving evidence of the events that changed those values.

5.2 Main Database Tables

The database contains both **master tables** and **transactional tables**. Master tables store relatively stable information, such as users, roles, suppliers, customers, categories and inventory items. Transactional tables store operational activity, such as purchases, sales, returns, adjustments, stock movements and audit events.

This separation was important because the system needed to store the current stock position while also preserving the history behind each change. For example, the InventoryItems table stores the current quantity of an item, but the StockLedger table records the movement that caused the quantity to change.

Table	Purpose
Users	Stores user account details, assigned role and active status.
Roles	Stores role names used by the role-based access-control model.
AccessRequests	Stores requests from users who need system access.
InventoryItems	Stores item master data and the current stock state.
Categories	Stores item category details used to organise inventory.
Suppliers	Stores supplier details used in the purchase workflow.
Customers	Stores customer details used in the sales workflow.
PurchaseOrders	Stores purchase order header records.
PurchaseOrderItems	Stores purchase order line items.
SalesOrders	Stores sales order header records.
SalesOrderItems	Stores sales order line items.
Returns	Stores sales return and purchase return records.
StockAdjustments	Stores manual stock correction records.
StockLedger	Stores unified stock movement records.
AuditLogs	Stores system audit events and security-relevant activity.

Table 5.1 – Main Database Tables

The most important tables for stock integrity are InventoryItems, PurchaseOrderItems, SalesOrderItems, Returns, StockAdjustments and StockLedger. These tables work together so that stock changes can be connected to a reason, reference number and user action rather than being stored as unexplained quantity changes.

5.3 InventoryItems Table

The InventoryItems table stores the core data for each stock item. It acts as the main item master table and is used by the inventory, purchase, sales, returns, adjustment and stock movement modules. Important fields include

Field	Purpose
ItemId	Primary identifier for each inventory item.
SKU	Stock keeping unit used for searching and identifying items.
ItemName	The display name of the product.
CategoryId	Links the item to its category.
CurrentQuantity	Stores the current available stock quantity.
MinimumLevel	Defines the stock level at which the item is considered low.
UnitPrice	Stores the purchase/unit cost value used in purchase-related records.

SellingPrice	Stores the minimum configured selling price for sales.
LowStockAlertSent	Prevents repeated low-stock email alerts for the same item.
CreatedBy / UpdatedBy	Records which user created or updated the item.
CreatedDate / UpdatedDate	Records when the item was created or updated.

Table 5.2 – Key Fields in the inventoryitems table

The SellingPrice field is important because it supports one of the main sales rules in the system: users cannot sell an item below its configured selling price. This prevents accidental under-pricing and protects the business rule that sales must respect the item’s stored selling value.

The CurrentQuantity field is also critical, but it is not treated as a value that users can freely change. Instead, it is updated through controlled workflows such as purchase approval, sales approval, returns and stock adjustments. This design reduces the risk of unexplained stock changes.

The MinimumLevel and LowStockAlertSent fields support the low-stock alert feature. When stock falls below the minimum level after a sales approval, the system can send an email alert to Admin and SuperAdmin users. The LowStockAlertSent flag helps prevent the same low-stock warning from being repeatedly sent for the same item.

The use of SKU also improves usability because stock items can be identified consistently across the inventory page, purchase items, sales items, returns, adjustments and stock movement history.

5.4 StockLedger Table

The StockLedger table is the operational stock movement history of the system. It records stock changes from different workflows using one shared structure. This table is important because it allows the system to answer not only **what the current stock level is**, but also **why that stock level changed**.

Important fields include:

Field	Purpose
LedgerId	Primary identifier for each stock movement record.
ItemId	Links the movement to the inventory item affected.
TransactionType	Identifies the workflow that caused the movement.
ReferenceId	Links the movement to the original transaction record where applicable.
ReferenceNumber	Provides a readable reference such as purchase, sales, return or adjustment number.
PartyName	Stores the related supplier, customer or system source.
Quantity	Records the quantity moved.
MovementType	Shows whether the movement increased or decreased stock.
CreatedBy	Records the user responsible for the movement.
CreatedDate	Records when the movement was created.

Table 5.3 – Key Fields in the StockLedger table

The TransactionType field identifies the source of the movement. In this project, the main transaction types are:

Transaction Type	Meaning
PURCHASE	Stock increased after an approved purchase order.

SALE	Stock decreased after an approved sales order.
SALES_RETURN	Stock increased after a customer returned sold items.
PURCHASE_RETURN	Stock decreased after items were returned to a supplier.
ADJUSTMENT	Stock was manually increased or decreased for correction purposes.

Table 5.4 – StockLedger transaction type

The MovementType field identifies the direction of the stock movement. IN means the stock quantity increased, while OUT means the stock quantity decreased. This simple distinction makes the stock history easier to understand across different workflows.

For example, a purchase approval creates a PURCHASE transaction with movement type IN, while a sales approval creates a SALE transaction with movement type OUT. A sales return also creates an IN movement because stock is coming back into the warehouse, whereas a purchase return creates an OUT movement because stock is leaving the warehouse and going back to the supplier.

This structure allows the Stock Movement History page to display all stock activity in one place. Without this table, the user would need to check separate purchase, sales, return and adjustment records to understand how stock changed. The unified ledger therefore improves traceability, supports audit review and makes the system more suitable for a controlled warehouse environment.

One design decision I made was to keep the StockLedger focused on **stock movement**, not financial accounting. It does not record full payment or money transaction history. For this reason, the page title “Stock Movement History” is more accurate than “Stock Ledger” from a user-facing perspective, although the database table name StockLedger is still suitable because it acts as a stock movement ledger.

5.5 AuditLogs Table

The AuditLogs table stores accountability evidence for important user and system actions. This was included because the system is not only responsible for changing stock, but also for showing **who did what, when they did it, where it happened, and whether the action was successful or risky**.

Important fields include:

Field	Purpose
LogId	Primary identifier for each audit record.
UserId	Links the action to the user who performed it.
EventCategory	Groups the event, such as authentication, stock, user management or system error.
EventType	Describes the specific event, such as page access, sale approval or failed access attempt.
Description	Stores a readable explanation of the action.
PageName	Records the page where the action occurred.
Device	Stores the device type detected during the request.

Browser	Stores browser information.
IPAddress	Records the IP address connected to the request.
Severity	Classifies the importance of the event, such as information, warning or critical.
TimeStamp	Records when the event happened.

Table 5.5 – Key Fields in the AuditLogs Table

The audit log supports the Auditor role because auditors need visibility of important system activity without being allowed to change operational data. It also helps Admin and SuperAdmin users investigate errors, unauthorised access attempts and unusual stock-related activity.

The AuditLogs table is populated through the AuditLoggerService and AuditLoggerDAL classes. These classes call the sp_AuditLogged_Insert stored procedure, which inserts the audit event into the database. This means pages do not directly insert audit records themselves. Instead, they call a shared logging service, which keeps the logging approach consistent across the system.

Logging is grouped into the following areas in the implemented System Audit Logs page:

Logged Area	Example Events
AUTH / Authentication	Login success, login failure and authentication-related activity.
STOCK / Inventory	Inventory access, purchase item changes, sales item changes, stock approval actions and stockrelated operations.
USER_MANAGEMENT / User Management	Creating, updating or deleting users, customers and suppliers where applicable.
SYSTEM_USAGE / System Usage	Page access events, opening major modules and normal system navigation.
SYSTEM_ERROR / System Errors	Failed operations, validation failures, exception handling and workflow errors.
UNAUTH / Unauthorised Access	Attempts to access restricted pages or perform actions without permission.

Table 5.6 – Audit log categories used in the system

This design improves accountability because important actions are not invisible. For example, if stock is reduced through a sale approval or manual adjustment, the system can show the stock movement in the StockLedger table and the user action in the AuditLogs table. The two tables therefore support different but related forms of traceability: one explains the stock movement, and the other explains the user/system activity.

5.6 Stored Procedures

Stored procedures are used to centralise database operations and keep SQL logic out of the ASP.NET pages. The application sends values to stored procedures through parameterised ADO.NET commands. This supports maintainability because database operations can be updated in one place, and it also reduces the risk of unsafe SQL construction.

Stored Procedure	Purpose
------------------	---------

sp_Customer_Insert	Inserts a new customer.
sp_Customer_GetAll	Retrieves customer records.
sp_SearchAll_Customer	Searches customers.
SP_SuggestionCustomer	Provides customer autocomplete suggestions.
sp_Sales_Create	Creates a sales order.
SP_sales_GetAll	Retrieves sales orders.
SP_Search_salesOrders	Searches sales orders.
SP_sales_AutoComplete	Provides sales search suggestions.
sp_HandleReturn	Handles sales and purchase returns.
sp_StockAdjustment	Handles stock adjustment IN and OUT.
sp_GetStockLedgerFiltered	Retrieves filtered stock movement history.
SP_SuggestStockLedger	Provides stock ledger autocomplete suggestions.
sp_AuditLogged_Insert	Inserts audit log records.
sp_Search_AuditLogs	Searches audit logs.
SP_SuggestAuditLogs	Provides audit log autocomplete suggestions.

Table 5.7— Example Stored Procedures used in system

The most important stored procedures are those that change stock or record traceability. These include the procedures used for purchase approval, sales approval, return handling, stock adjustment, stock movement retrieval and audit insertion.

A key design decision was to avoid spreading database update logic directly across the Web Forms pages. For example, the Returns page does not manually update the inventory table from the page code. Instead, the page collects the selected return type, reference order, item, quantity and reason, then passes the operation through the service and DAL layers to the stored procedure responsible for handling the return. This improves reliability because the stock update and return record are handled in a controlled workflow.

The same idea is used for stock adjustment. The application checks user input and role permissions, but the actual stock-changing operation is handled through the database procedure. This helps protect the system from inconsistent changes, such as allowing an adjustment OUT that would make stock negative.

6. Implementation

6.1 Technology Stack

The project was implemented using the following technologies:

Component	Technology
Web framework	ASP.NET Web Forms
Programming language	C#
Database	SQL Server
Data access	ADO.NET
Database operations	Stored procedures
UI controls	ASP.NET server controls and AjaxControlToolkit
Validation	ASP.NET validators, custom validators and service validation
Email	SMTP-based email service
Export	CSV and Excel export using server response generation

Table 6.1 – Technology Stack

ASP.NET Web Forms was used because it supports server-side event handling, built-in controls, validators and a page lifecycle model suitable for a data-driven business application. C# was used for the page logic, service classes, workflow classes and data access code.

SQL Server was used to store user accounts, inventory items, purchase records, sales records, returns, stock adjustments, stock movement history and audit logs. The application communicates with the database using ADO.NET. In this project, ADO.NET is the part of .NET used to open SQL connections, create SqlCommand objects, pass parameters, execute stored procedures and fill DataTable objects through SqlDataAdapter.

Stored procedures were used for the main database operations because they keep SQL logic centralised and reduce the need to write SQL directly inside the application pages. This is visible across the system in procedures such as sp_Sales_Create, sp_HandleReturn, sp_StockAdjustment, sp_GetStockLedgerFiltered and sp_AuditLogged_Insert.

The AjaxControlToolkit was used to improve usability, especially through autocomplete controls and modal popup forms. This was useful in workflows where the user should select a valid database record rather than type free text. For example, the returns page uses cascading autocomplete so that a user first selects a customer or supplier, then selects the related order, and then selects an item from that order.

Validation was implemented at multiple levels. ASP.NET validators check required fields, ranges and formats on the page. Custom validators check whether autocomplete selections have produced valid hidden IDs. Service-layer validation applies business rules such as preventing negative stock, blocking sales below the configured selling price and preventing duplicate sales items with different prices.

The system also includes an SMTP-based low-stock email service and export functionality for audit logs. The export feature uses server response generation to produce CSV and Excel files from audit log data. This supports the accountability requirement because Admin, SuperAdmin and Auditor users can review and extract audit evidence when needed.

6.2 Authentication and Session Validation

The system requires users to log in before accessing protected pages. The login page collects the user's email address and password and validates the credentials before allowing access to the internal system. After a successful login, key user information such as the user ID, user name and role is stored in session variables. These session values are then used across the application to identify the current user and apply role-based rules.

Internal pages call `ValidateSession()` to confirm that the user is authenticated before the page is loaded. This is important because users should not be able to access internal pages by directly entering a URL in the browser. If the session is missing or invalid, the user is redirected back to the login page.

The system also calls `RequireRole(...)` on protected pages to restrict access by role. This provides backend protection in addition to hiding menu options in the interface. Menu visibility alone is not secure because a user could still attempt to open a restricted page manually. Backend role validation ensures that direct access attempts are also controlled. For example, pages use role checks such as:

```
RequireRole("SuperAdmin", "Admin", "Staff", "Auditor");
```

This means that only users with one of the listed roles can access that page. Other pages use stricter rules. For example, the Stock Adjustment page is restricted to SuperAdmin, Admin and Auditor roles, while operational approval actions are hidden from Staff and Auditor users where required.

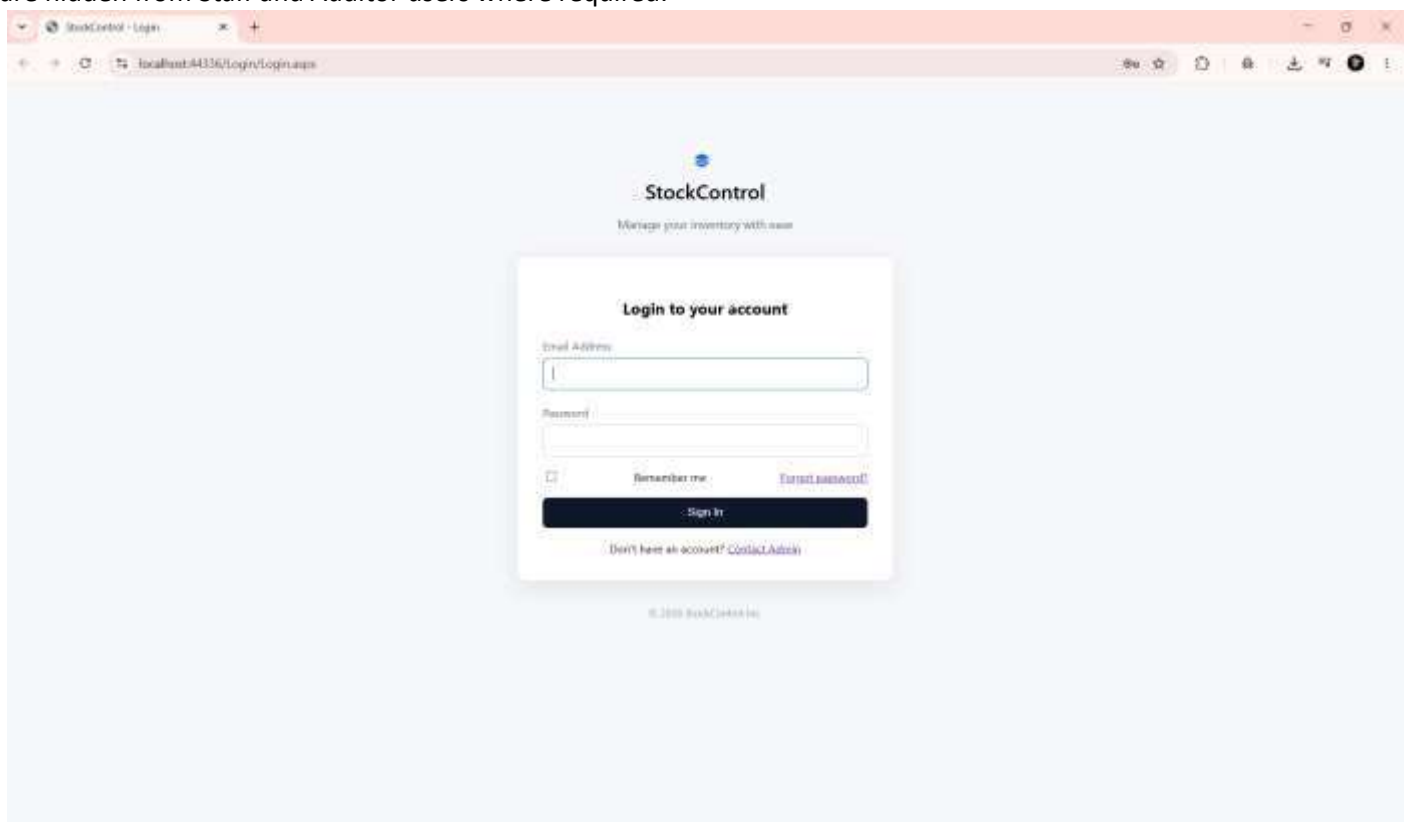


Figure 6.1 – Login Page for user authentication

Figure 6.1 shows the login page used to authenticate users before they can access the Stock Control System. The page also includes “Remember me”, “Forgot password?” and “Contact Admin” options, which support a more complete access workflow rather than only basic login.

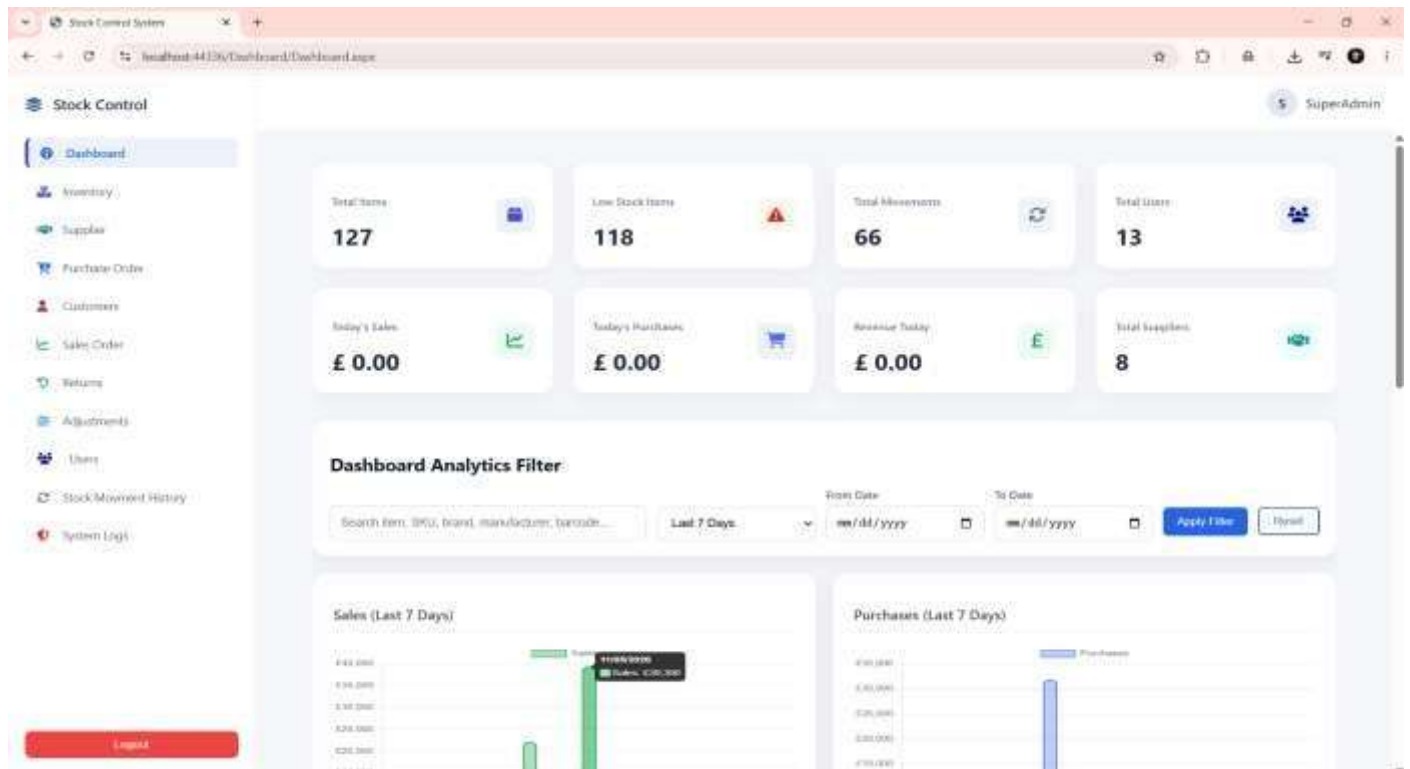


Figure 6.2– Dashboard Displayed after successful login

Figure 6.2 shows the dashboard reached after successful authentication. The dashboard confirms that the user has entered the protected part of the system, where navigation and available actions are then controlled according to the user’s assigned role.

6.3 Access Requests and User Management

The access request module allows a person to request system access instead of directly creating their own privileged account. This is safer than open registration because the system handles stock records, user permissions and auditsensitive actions. A new user request can be reviewed by an authorised user before access is granted, which reduces the risk of unknown users entering the system.

The user management module allows authorised users to manage existing accounts, roles and active status. This supports the four-role model used across the system: SuperAdmin, Admin, Staff and Auditor. Role assignment is important because the user’s role determines which pages, buttons and workflow actions are available.

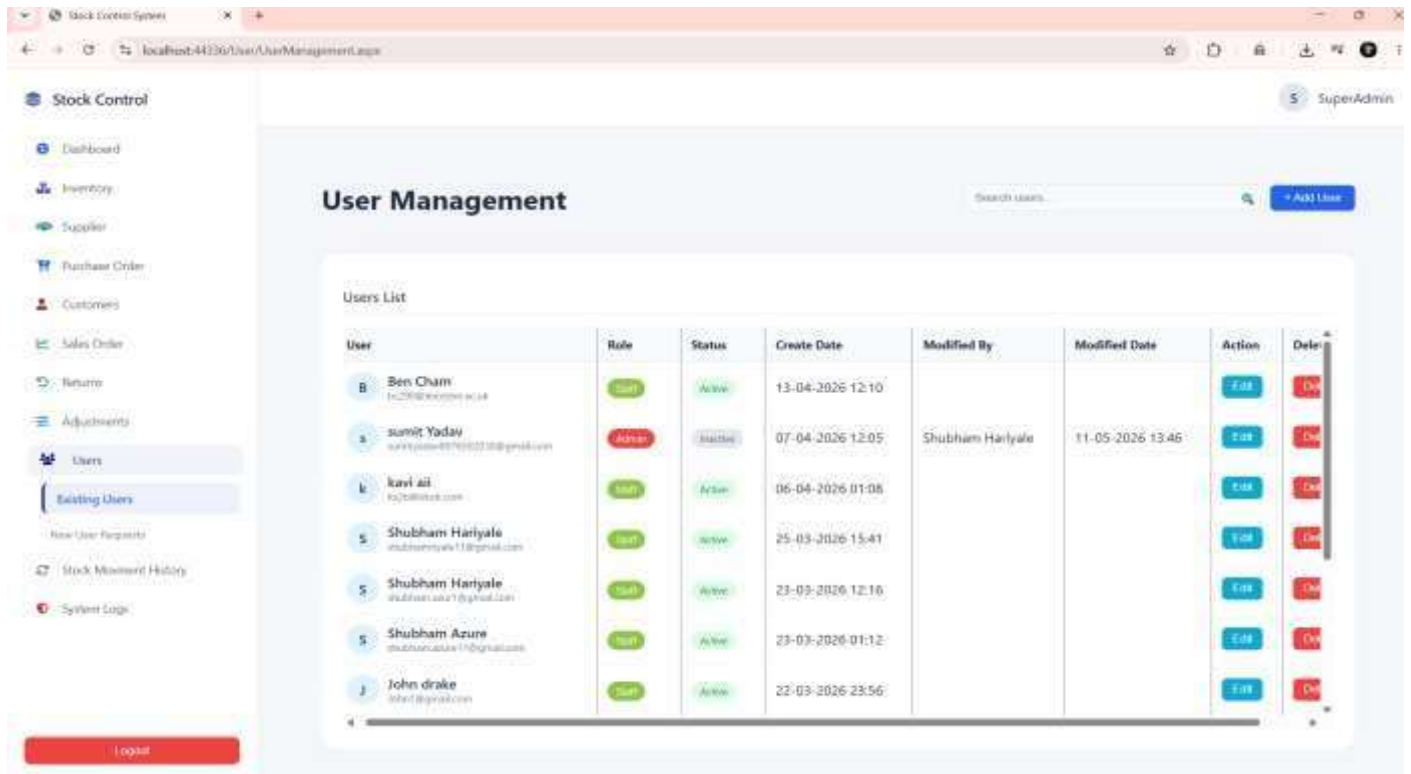


Figure 6.3 – User Management page for managing roles and account status

Figure 6.3 shows the implemented User Management page. The page displays user identity, role, account status, creation date, modification details and available actions. This supports controlled administration because authorised users can review whether an account is active and which role has been assigned.

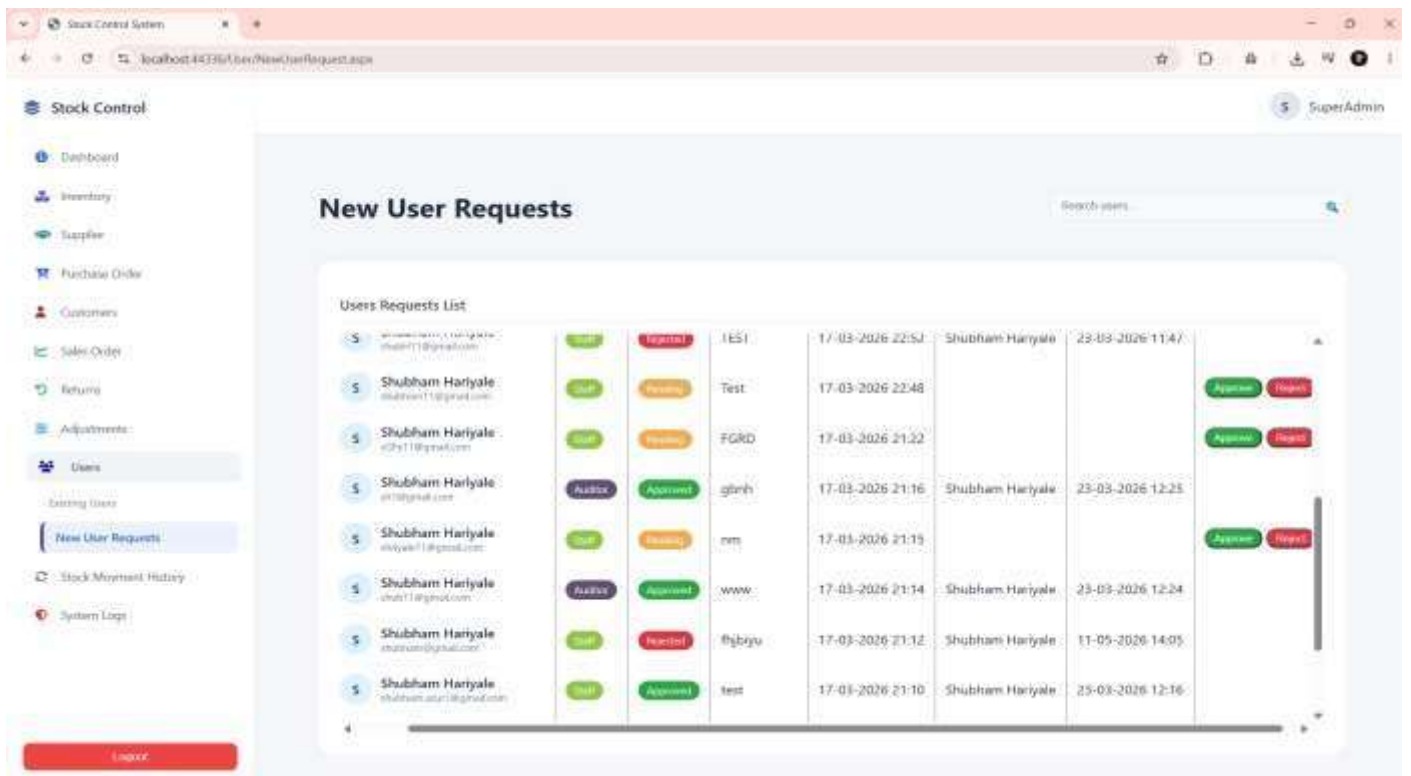


Figure 6.4 – Access Request Page for approving and rejecting new users

Figure 6.4 shows the New User Requests page. Requests can be reviewed with their requested role, status, reason and processing information. The approve and reject buttons show that access is not automatically granted. This supports the security requirement because user access is controlled through an approval workflow rather than open selfregistration.

This design also supports auditability. Access decisions are linked to user roles and can be reviewed alongside the system audit logs. In practice, this means the system can show not only which users exist, but also how access was requested and controlled.

6.4 Inventory Management

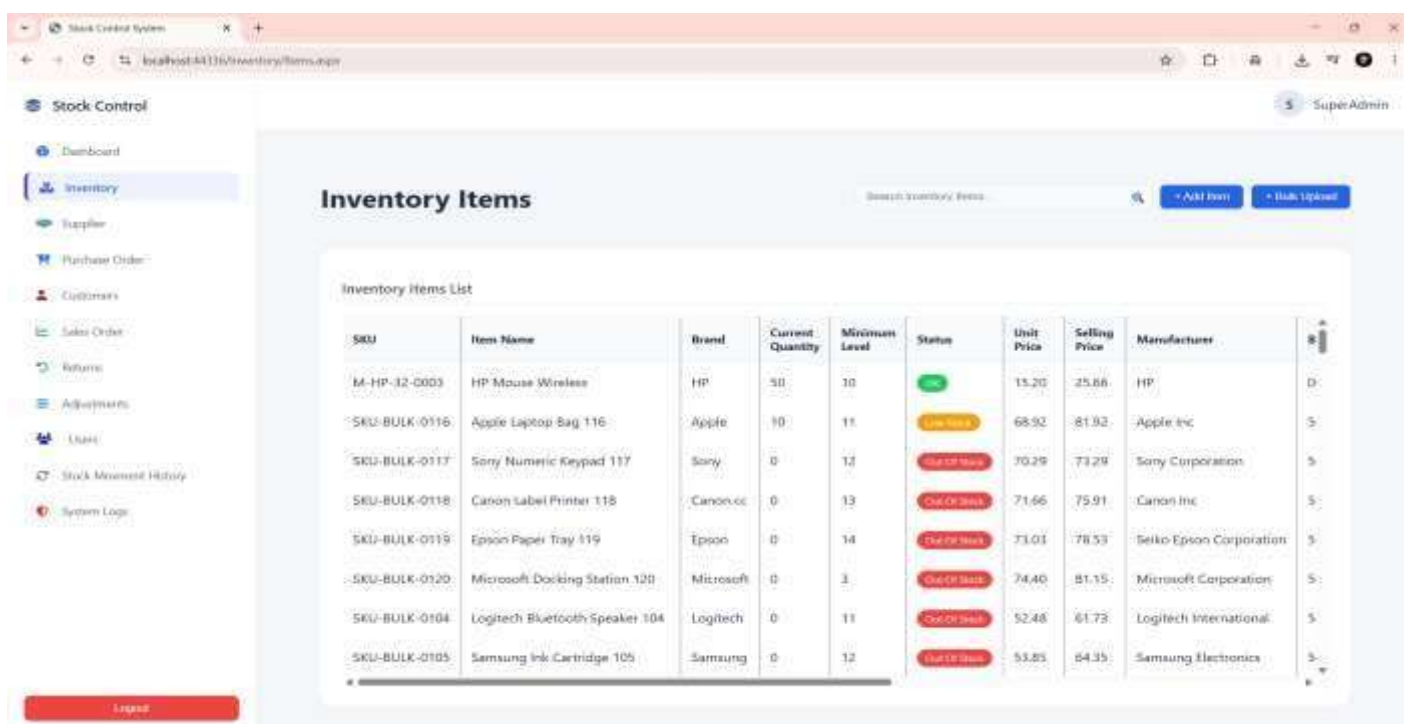
The inventory module manages the product records used by the rest of the system. Each inventory item can store details such as SKU, item name, brand, manufacturer, barcode, description, unit price, selling price, warranty information, current quantity and minimum stock level. These fields are important because the inventory module is not isolated; the same item records are later used by purchase orders, sales orders, returns, stock adjustments and stock movement history.

A key implementation decision was to treat the inventory item as the master product record, while stock quantity changes are handled through controlled workflows. This means the item page provides product information and current stock status, but the stock value is mainly affected by approved purchases, approved sales, returns and stock adjustments. This helps prevent unexplained stock changes.

The inventory page also displays stock status based on the current quantity and minimum level:

Status	Meaning
Out of Stock	The current quantity is zero.
Low Stock	The current quantity is greater than zero but less than or equal to the minimum level.
OK	The current quantity is above the minimum level.

Table 6.2 – Inventory stock status rules



SKU	Item Name	Brand	Current Quantity	Minimum Level	Status	Unit Price	Selling Price	Manufacturer
M-HP-32-0003	HP Mouse Wireless	HP	50	10	OK	15.20	25.88	HP
SKU-BULK-0116	Apple Laptop Bag 116	Apple	10	11	Low Stock	68.92	81.92	Apple Inc
SKU-BULK-0117	Sony Numeric Keypad 117	Sony	0	12	Out of Stock	70.29	73.29	Sony Corporation
SKU-BULK-0118	Canon Label Printer 118	Canon Inc	0	13	Out of Stock	71.66	75.91	Canon Inc
SKU-BULK-0119	Epson Paper Tray 119	Epson	0	14	Out of Stock	73.03	78.53	Seiko Epson Corporation
SKU-BULK-0120	Microsoft Docking Station 120	Microsoft	0	3	Out of Stock	74.40	81.15	Microsoft Corporation
SKU-BULK-0104	Logitech Bluetooth Speaker 104	Logitech	0	11	Out of Stock	52.48	61.73	Logitech International
SKU-BULK-0105	Samsung Ink Cartridge 105	Samsung	0	12	Out of Stock	53.85	64.35	Samsung Electronics

Figure 6.5 – Inventory Management Page showing stock status and product details

Figure 6.5 shows the implemented Inventory Items page. The page displays each item with its SKU, item name, brand, current quantity, minimum level, stock status, unit price, selling price and manufacturer. The colour-coded status badges make stock condition visible without needing to inspect every quantity manually. For example, items with zero quantity are shown as **Out of Stock**, while items close to or below their minimum level are shown as **Low Stock**.

This page provides the product foundation for the wider stock-control workflow. Purchase approval increases the quantity of these items, sales approval decreases it, returns reverse previous purchase or sales movement, and stock adjustments correct stock differences. Because of this, the inventory page acts as the central reference point for product data, while the Stock Movement History page explains how each quantity changed over time.

6.5 Supplier and Customer Management

The supplier and customer modules manage the external parties involved in stock workflows. Suppliers are used in the purchase workflow, while customers are used in the sales and sales return workflows. These modules are important because purchase and sales records should not rely on manually typed party names only; they should be linked to controlled records in the database.

Both modules follow a similar implementation pattern. Each page includes a searchable list, autocomplete suggestions, modal forms for creating records, validation rules and role-based visibility for actions such as editing or deleting. This keeps the interface consistent with the rest of the application and reduces the amount of learning required when users move between pages.

The supplier module stores supplier details such as supplier name, contact person, email, phone number, address and country. Supplier records are later used when creating purchase orders and purchase returns.

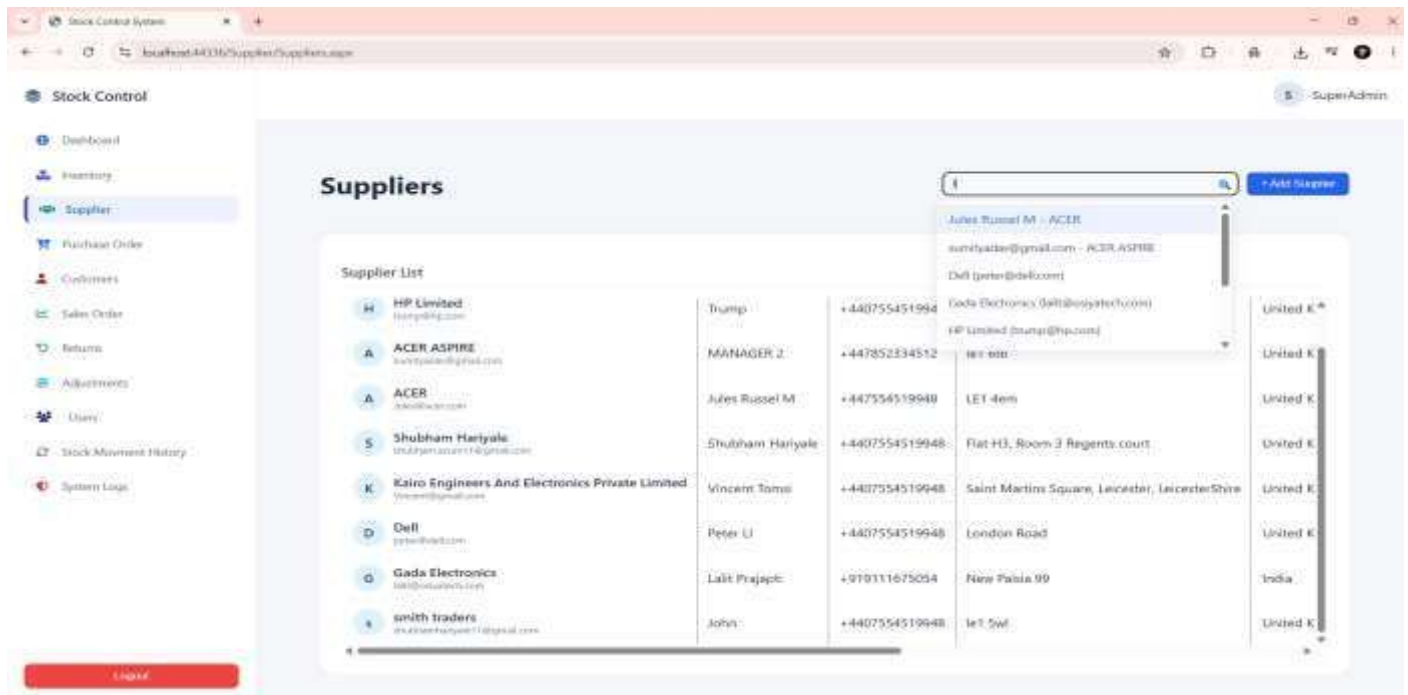


Figure 6.6 – Supplier Management Page with search autocomplete suggestion

Figure 6.6 shows the Supplier Management page. The page includes a supplier list, search bar, autocomplete suggestions and an Add Supplier button. The autocomplete suggestions help the user quickly locate existing suppliers and reduce typing errors when working with supplier records.

The customer module follows the same structure but supports the sales side of the system. Customer records are used when creating sales orders and when processing sales returns. The module stores customer information such as contact person, phone number, address, created by, created date, updated by and updated date.

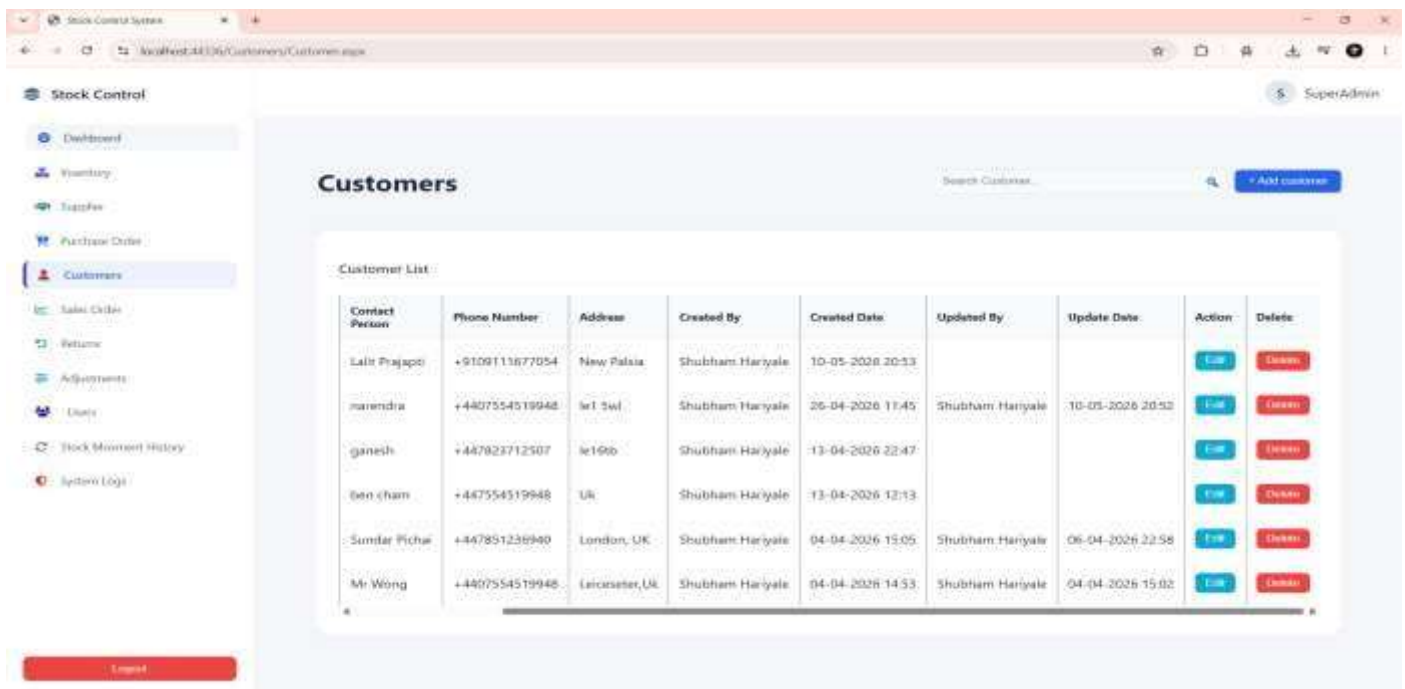


Figure 6.7 – Customer Management Page with editable customer records

Figure 6.7 shows the Customer Management page. The page displays customer records in a GridView and provides actions for editing and deleting records where the current user role allows it. This supports day-to-day data management while still keeping the customer data structured and searchable.

Validation is used in both modules to improve data quality. Required fields prevent incomplete records, while phonenummer validation helps avoid invalid contact details. Autocomplete and hidden ID values are also used in several workflows so that later operations rely on selected database records rather than uncontrolled typed text. This is especially important in the sales, purchase and returns workflows, where selecting the wrong party could create incorrect stock history.

6.6 Purchase Order Implementation

The purchase order module supports the process of recording incoming stock from suppliers in a controlled way. A user first creates a purchase order header linked to a supplier, and then adds one or more purchase items to that order. At this stage, the purchase remains in **Draft** status.

A key design decision in this project is that **stock is not increased when a purchase order is first created**. Instead, stock is increased **only when the purchase is approved** by an authorised user such as an Admin or SuperAdmin. This approach reduces the risk of inaccurate stock records caused by incomplete, abandoned or unverified purchase entries. It also reflects a more realistic warehouse process, where stock should only be counted once the order has been reviewed and confirmed.

The purchase items page allows the user to add products, enter quantity and price, and view the running order total. Once the purchase is complete and validated, the authorised user can approve it. Approval triggers the stock update workflow, which increases the quantity of each purchased inventory item, records the movement in the StockLedger table and creates an audit log entry.

This implementation is important because it links operational activity, stock integrity and accountability in one workflow. The page does not directly change stock simply because an item appears in a draft order. Instead, the stock update happens only through the approval logic, which makes the process safer and easier to audit.

Example Workflow Code

The following service-level method shows the simplified approval workflow:

```
public void ApprovePurchaseWithTransaction(int purchaseId, int userId)
{
    purchase.ApprovePurchase(purchaseId, userId);
    purchase.ResetLowStockAlert();

    logger.Log(
        userId,
        $"Purchase approved. PurchaseId: {purchaseId}",
        "PURCHASE APPROVED",
        "STOCK",
        "INFO"
    );
}
```

Listing 6.1 – Simplified Purchase Approval Workflow

This method shows how the system coordinates three related actions during purchase approval. First, it approves the purchase transaction. Second, it resets the low-stock alert flag where restocking means an earlier warning is no longer valid. Third, it records the approval event in the audit log. This is a good example of how the implementation combines stock control logic with traceability rather than treating them as separate concerns.

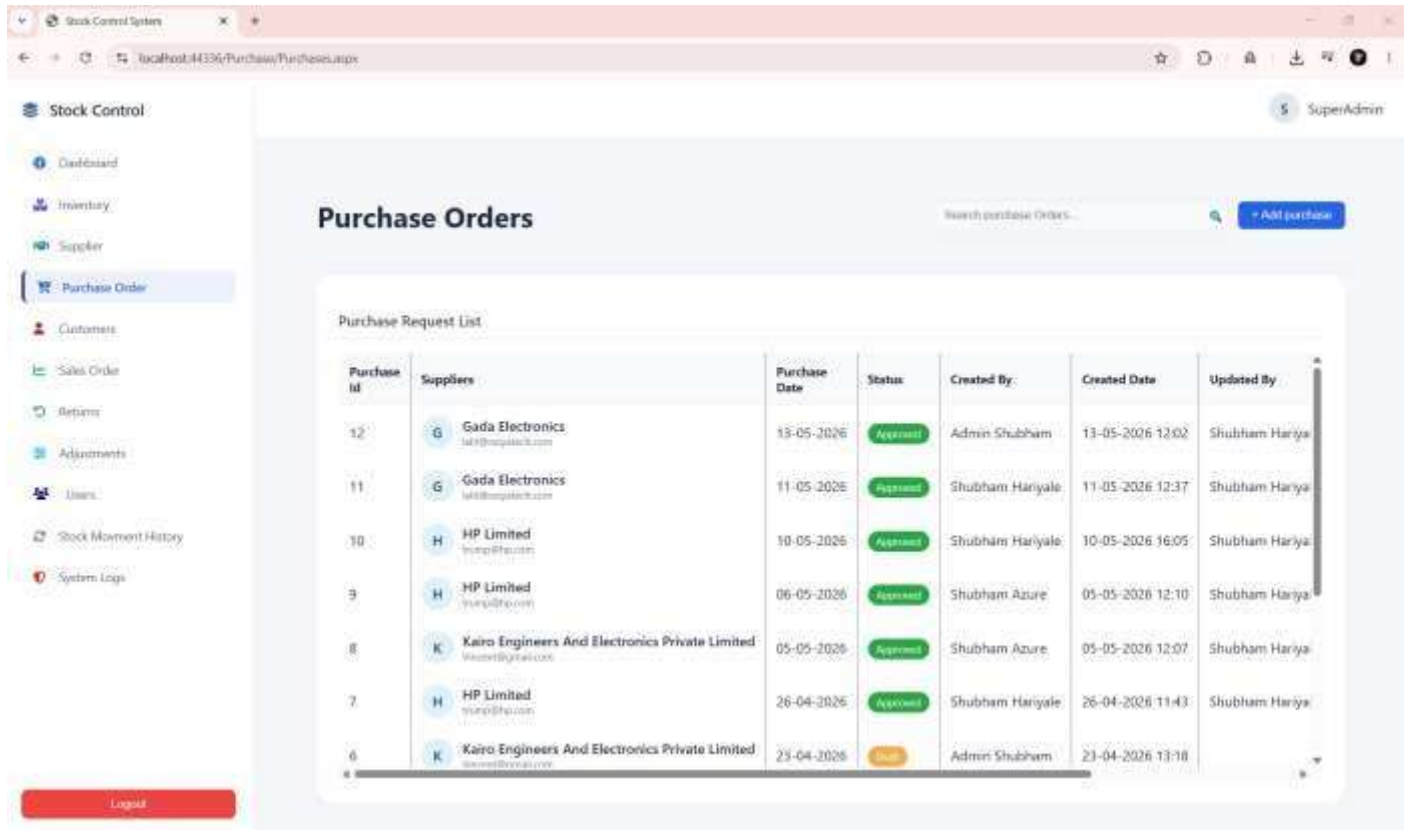


Figure 6.8 – Purchase Order List showing Draft and approved records

Figure 6.8 shows the Purchase Orders page. The page provides a searchable list of purchase records, displays the supplier, date, status, and created/updated metadata, and allows the user to begin a new purchase. The presence of both **Draft** and **Approved** statuses is important because it demonstrates that stock movement is controlled through workflow state rather than happening immediately at creation time.

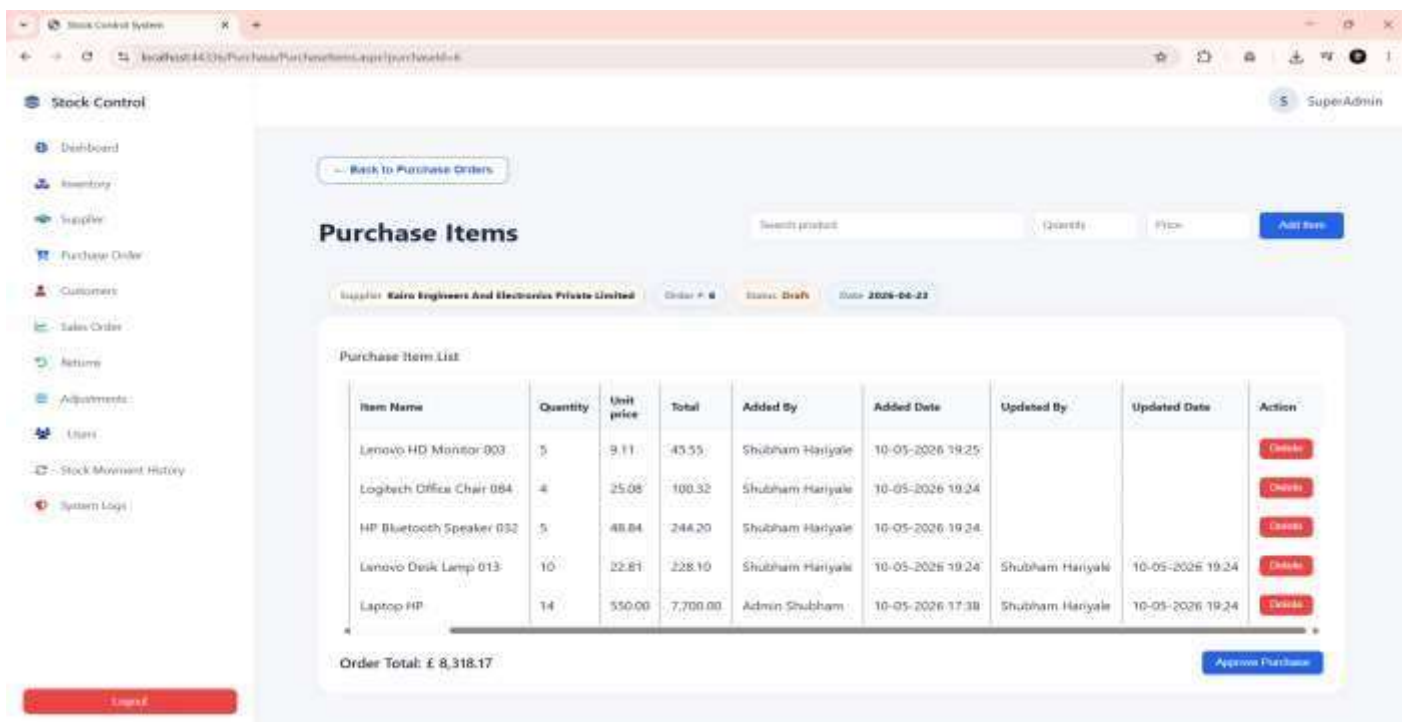


Figure 6.9 – Purchase Item page with list item, order total and approval action

Figure 6.9 shows the Purchase Items page for a selected purchase order. This page allows products to be added to the order and displays the quantity, unit price, total value, and audit-related metadata such as who added or updated the item. The **Approve Purchase** button is the key control point in this workflow, because it is the step that confirms the order and triggers the stock increase process.

6.7 Sales Order Implementation

The sales order module is one of the most important parts of the Stock Control System because it directly affects inventory reduction, price control and overselling prevention. A sales order is first created for a selected customer and remains in **Draft** status while items are added. Stock is not permanently reduced at this stage. The final stock deduction happens only when the sale is approved by an authorised user.

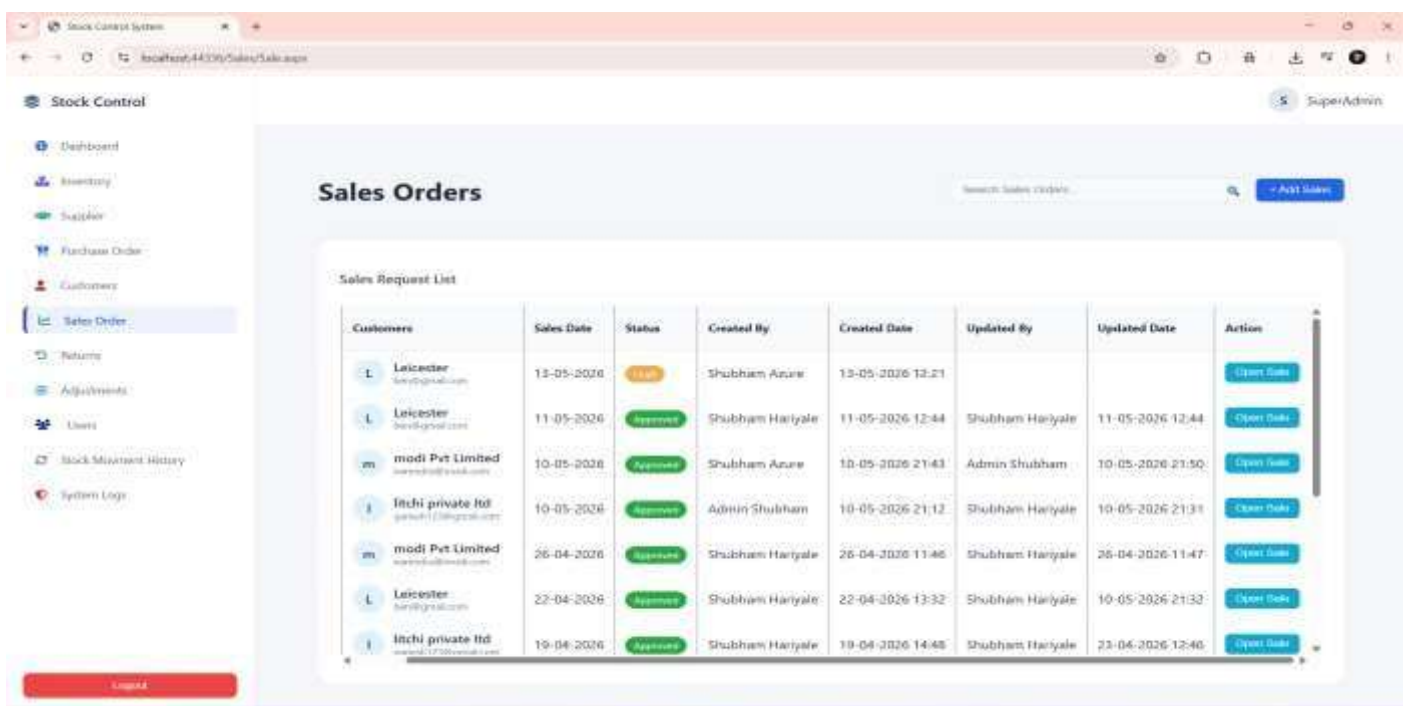
The sales item page uses autocomplete to select products. When a product is selected, the system stores the item ID in a hidden field and fills the price and available stock information. This is important because the system does not rely only on the visible product name typed by the user. It validates the selected item through its database ID.

The page also improves usability by showing available stock in the quantity placeholder. If an out-of-stock item is selected, the system displays **Out of Stock** and disables the Add Item button. This prevents the user from attempting an invalid sale from the interface.

However, interface validation is not treated as the only protection. The server-side logic also checks the requested quantity against available stock and considers the quantity already added to the same draft sales order. This means the system prevents the same order from repeatedly adding an item until it exceeds the available quantity. The detailed sales reservation logic is discussed later in **Listing 10.1** as one of the project's main technical achievements.

The sales workflow also enforces two important pricing and data-quality rules. First, the sale price cannot be lower than the configured selling price for the item. Second, the same item cannot be added to the same sales order using a different price. These checks help prevent accidental under-selling and inconsistent order lines.

When the sales order is approved, the system reduces inventory quantity, records a **SALE / OUT** movement in the StockLedger table and logs the approval event in the AuditLogs table. This means the system can show both what happened to the stock and who performed the action.



Customers	Sales Date	Status	Created By	Created Date	Updated By	Updated Date	Action
L Leicester leicester@leicester.com	13-05-2026	Draft	Shubham Azure	13-05-2026 12:21			Open Sale
L Leicester leicester@leicester.com	11-05-2026	Approved	Shubham Hariyale	11-05-2026 12:44	Shubham Hariyale	11-05-2026 12:44	Open Sale
m modi Pvt Limited modipvt@modipvt.com	10-05-2026	Approved	Shubham Azure	10-05-2026 21:43	Admin Shubham	10-05-2026 21:50	Open Sale
i Itchi private ltd itchipvt@itchipvt.com	10-05-2026	Approved	Admin Shubham	10-05-2026 21:12	Shubham Hariyale	10-05-2026 21:31	Open Sale
m modi Pvt Limited modipvt@modipvt.com	26-04-2026	Approved	Shubham Hariyale	26-04-2026 11:46	Shubham Hariyale	26-04-2026 11:47	Open Sale
L Leicester leicester@leicester.com	22-04-2026	Approved	Shubham Hariyale	22-04-2026 13:32	Shubham Hariyale	10-05-2026 21:32	Open Sale
i Itchi private ltd itchipvt@itchipvt.com	19-04-2026	Approved	Shubham Hariyale	19-04-2026 14:46	Shubham Hariyale	23-04-2026 12:46	Open Sale

Figure 6.10 – Sales Order List

Figure 6.10 shows the Sales Orders page, where users can view existing sales records, check their status and open a selected order. The status column is important because it separates draft sales from approved sales. This supports the project's design decision that stock should only be reduced after approval.

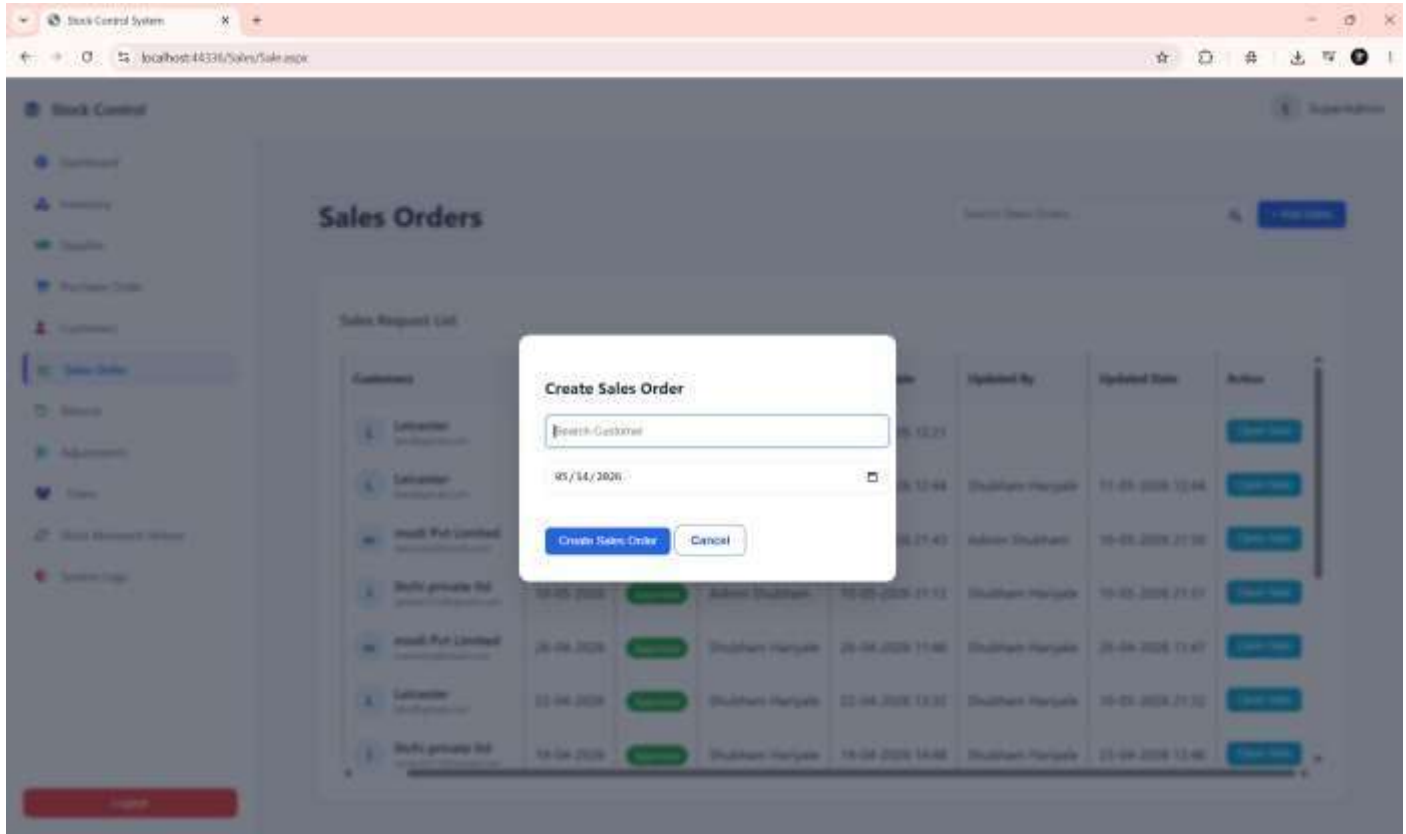


Figure 6.11 – Sales Order Creation modal

Figure 6.11 shows the modal used to create a new sales order for a customer. The form uses customer selection and date entry to create the order header before any sales items are added. This separates order creation from stockchanging activity.

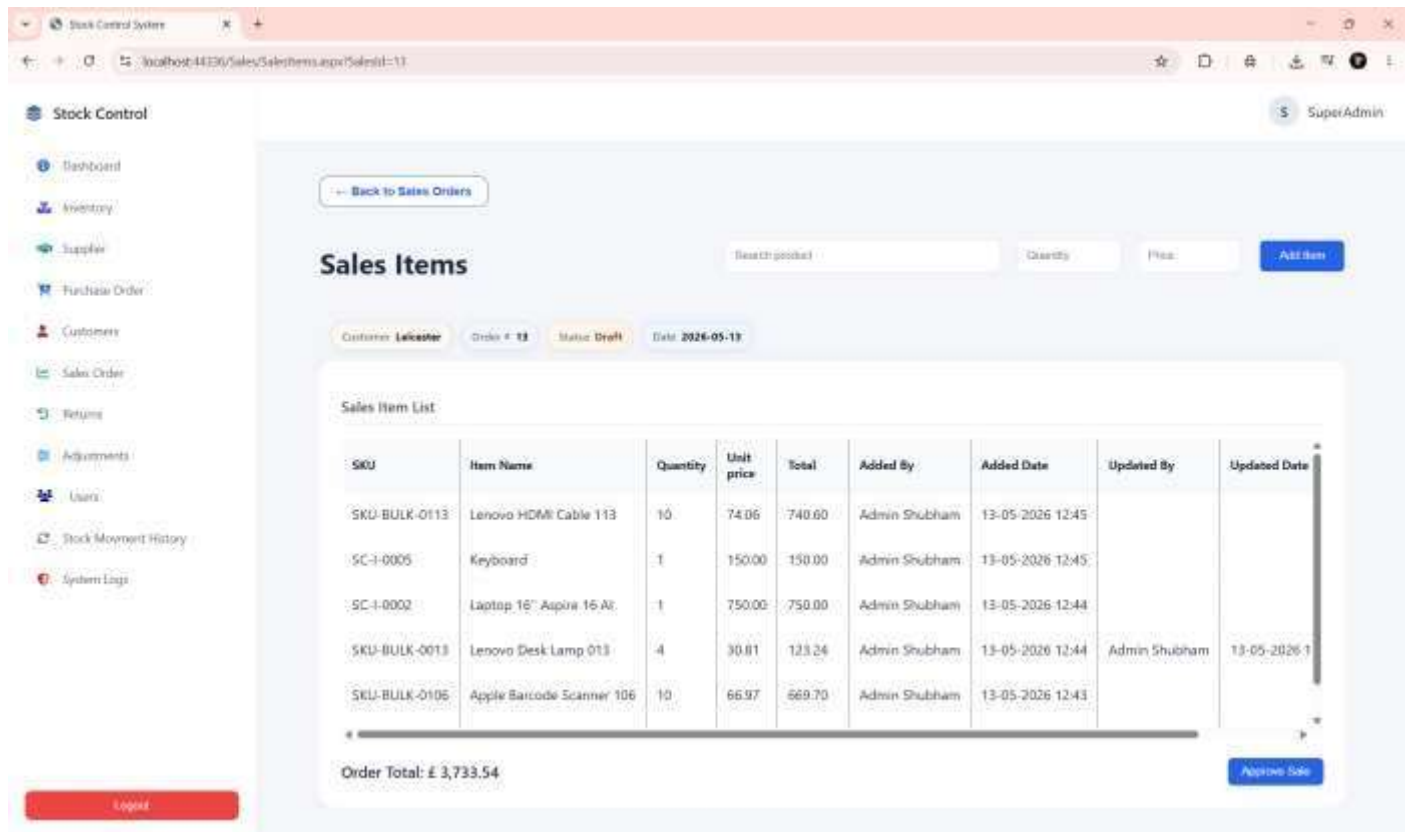


Figure 6.12 – Sales Order Item page and draft order details

Figure 6.12 shows the Sales Items page after items have been added to a draft order. The page displays the customer, order number, status, date, item list, quantity, unit price and order total. This screenshot demonstrates that a sale can be prepared before approval while still keeping final stock movement controlled.

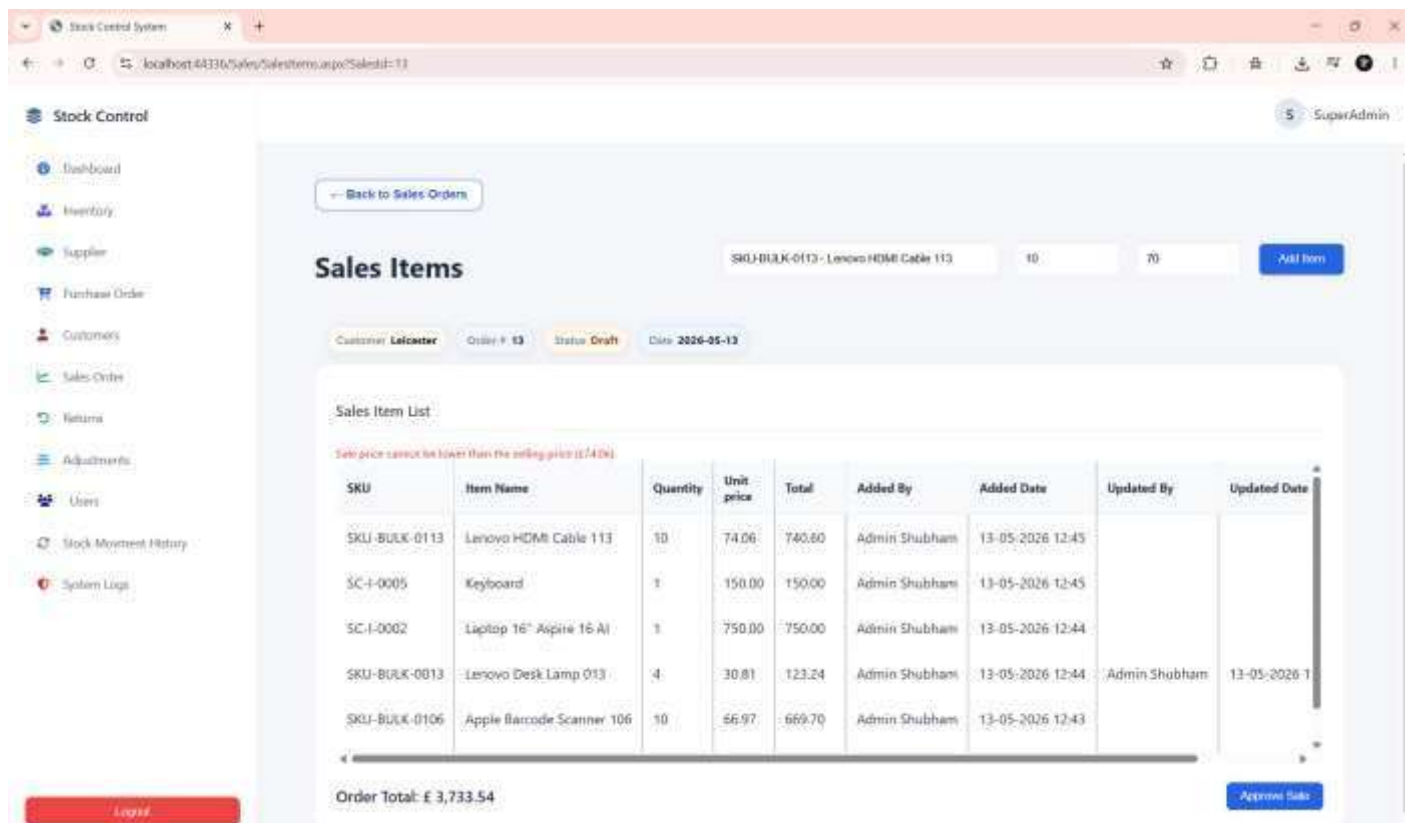
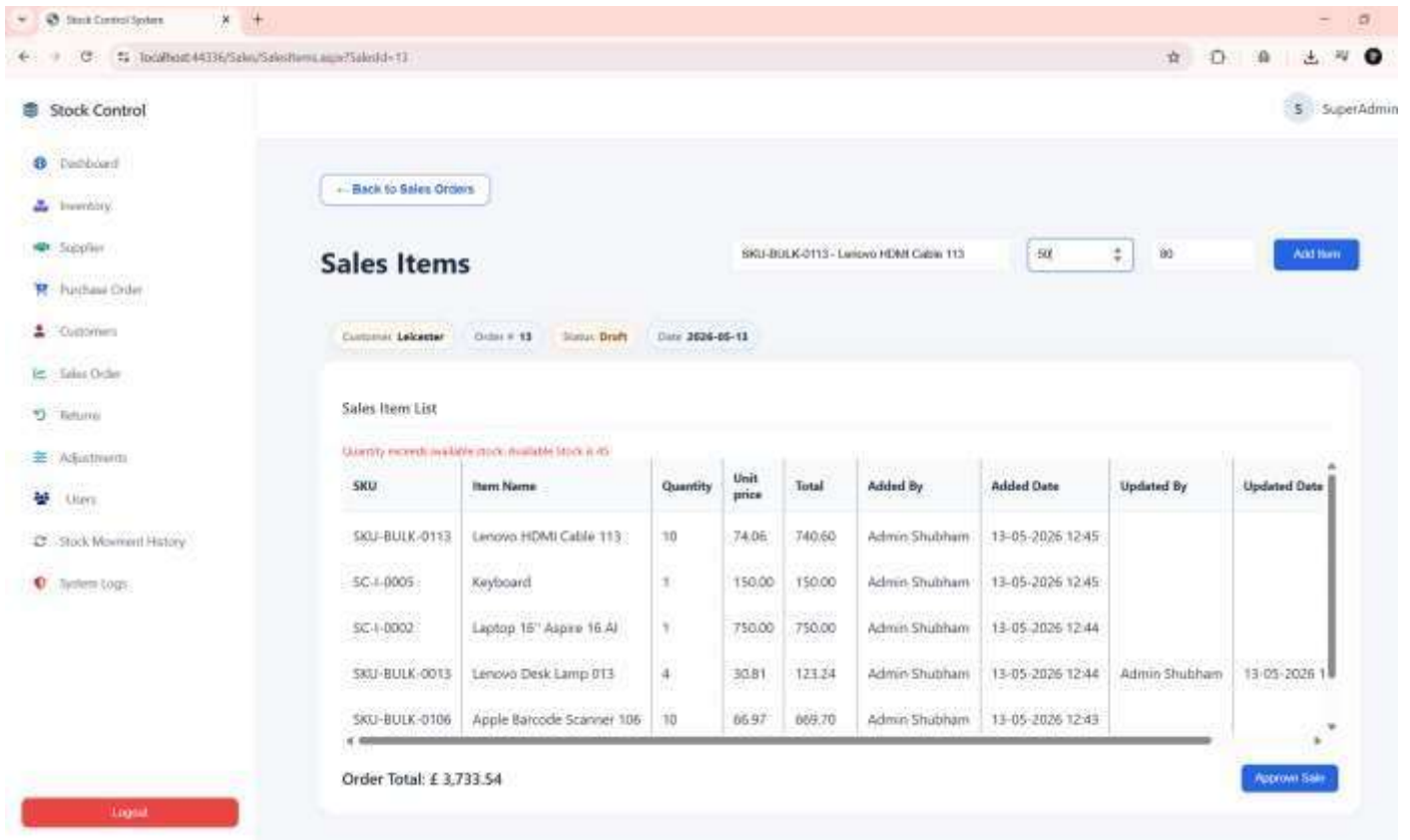


Figure 6.13 – Sales item Validation: below Selling Price

Figure 6.13 shows the system rejecting a sale price that is lower than the configured selling price. This validation protects the business rule that staff should not accidentally sell stock below the intended price.



Stock Control System

localhost:44336/Sales/SalesItems.aspx?SalesId=13

SuperAdmin

Back to Sales Orders

SKU-BULK-0113 - Lenovo HDMI Cable 113 50 80 Add Item

Customer: Leicester Order # 13 Status: Draft Date: 2026-05-13

Sales Item List

Quantity exceeds available stock: available stock is 40

SKU	Item Name	Quantity	Unit price	Total	Added By	Added Date	Updated By	Updated Date
SKU-BULK-0113	Lenovo HDMI Cable 113	10	74.06	740.60	Admin Shubham	13-05-2026 12:45		
SC-I-0005	Keyboard	1	150.00	150.00	Admin Shubham	13-05-2026 12:45		
SC-I-0002	Laptop 16" Aspire 16 AI	1	750.00	750.00	Admin Shubham	13-05-2026 12:44		
SKU-BULK-0013	Lenovo Desk Lamp 013	4	30.81	123.24	Admin Shubham	13-05-2026 12:44	Admin Shubham	13-05-2026 1
SKU-BULK-0106	Apple Barcode Scanner 106	10	66.97	669.70	Admin Shubham	13-05-2026 12:43		

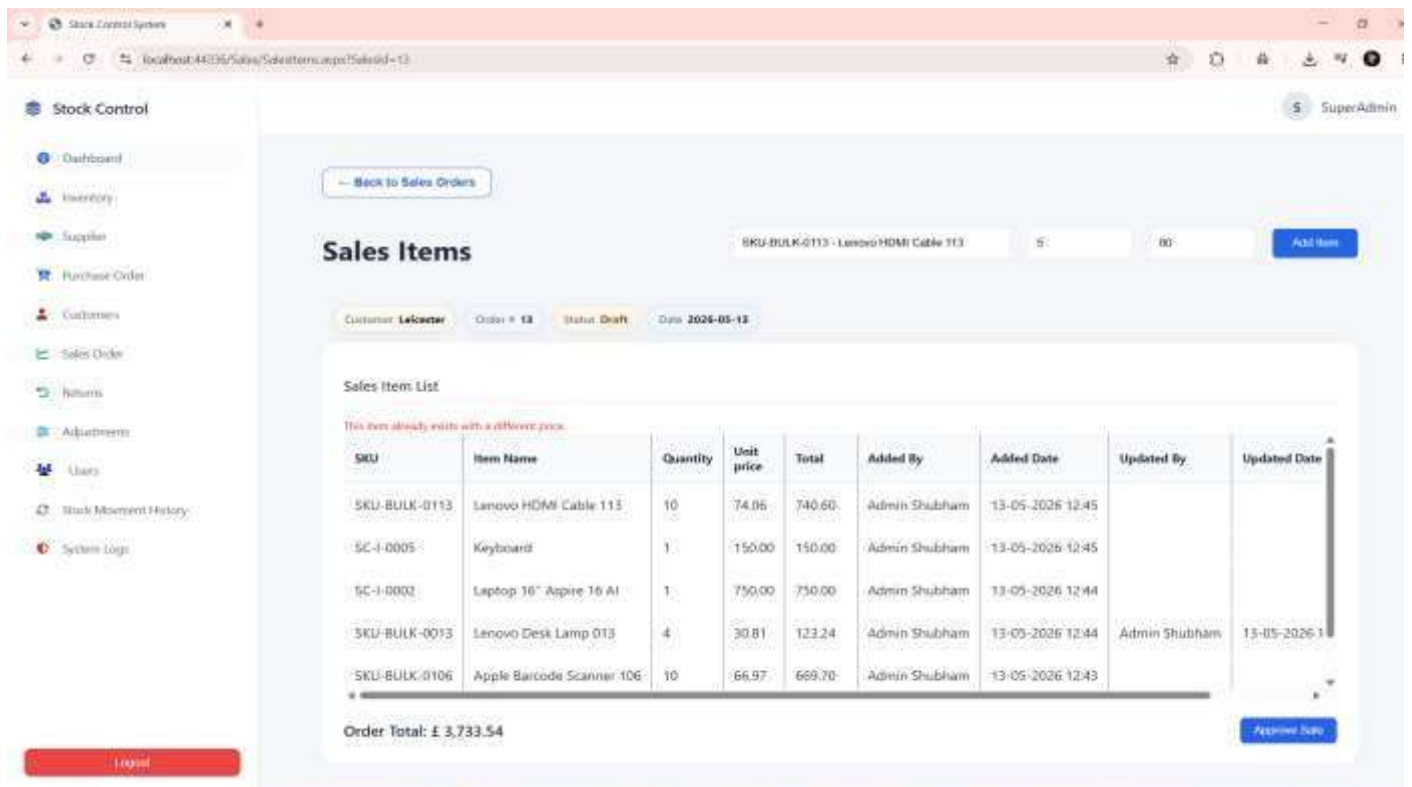
Order Total: £ 3,733.54

Approve Sale

Login

Figure 6.14 – Sales item Validation: Quantity Exceeds available stock

Figure 6.14 shows the system rejecting a quantity that exceeds available stock. This is one of the key integrity checks in the sales module because it helps prevent overselling.



Stock Control System

localhost:44336/Sales/SalesItems.aspx?SalesId=13

SuperAdmin

Back to Sales Orders

SKU-BULK-0113 - Lenovo HDMI Cable 113 5 80 Add Item

Customer: Leicester Order # 13 Status: Draft Date: 2026-05-13

Sales Item List

This item already exists with a different price.

SKU	Item Name	Quantity	Unit price	Total	Added By	Added Date	Updated By	Updated Date
SKU-BULK-0113	Lenovo HDMI Cable 113	10	74.06	740.60	Admin Shubham	13-05-2026 12:45		
SC-I-0005	Keyboard	1	150.00	150.00	Admin Shubham	13-05-2026 12:45		
SC-I-0002	Laptop 16" Aspire 16 AI	1	750.00	750.00	Admin Shubham	13-05-2026 12:44		
SKU-BULK-0013	Lenovo Desk Lamp 013	4	30.81	123.24	Admin Shubham	13-05-2026 12:44	Admin Shubham	13-05-2026 1
SKU-BULK-0106	Apple Barcode Scanner 106	10	66.97	669.70	Admin Shubham	13-05-2026 12:43		

Order Total: £ 3,733.54

Approve Sale

Login

Figure 6.15 – Sales item Validation: Duplicate item with Different Price

Figure 6.15 shows the system rejecting the same item being added to one sales order with a different unit price. This prevents inconsistent order data and keeps the sales total reliable.

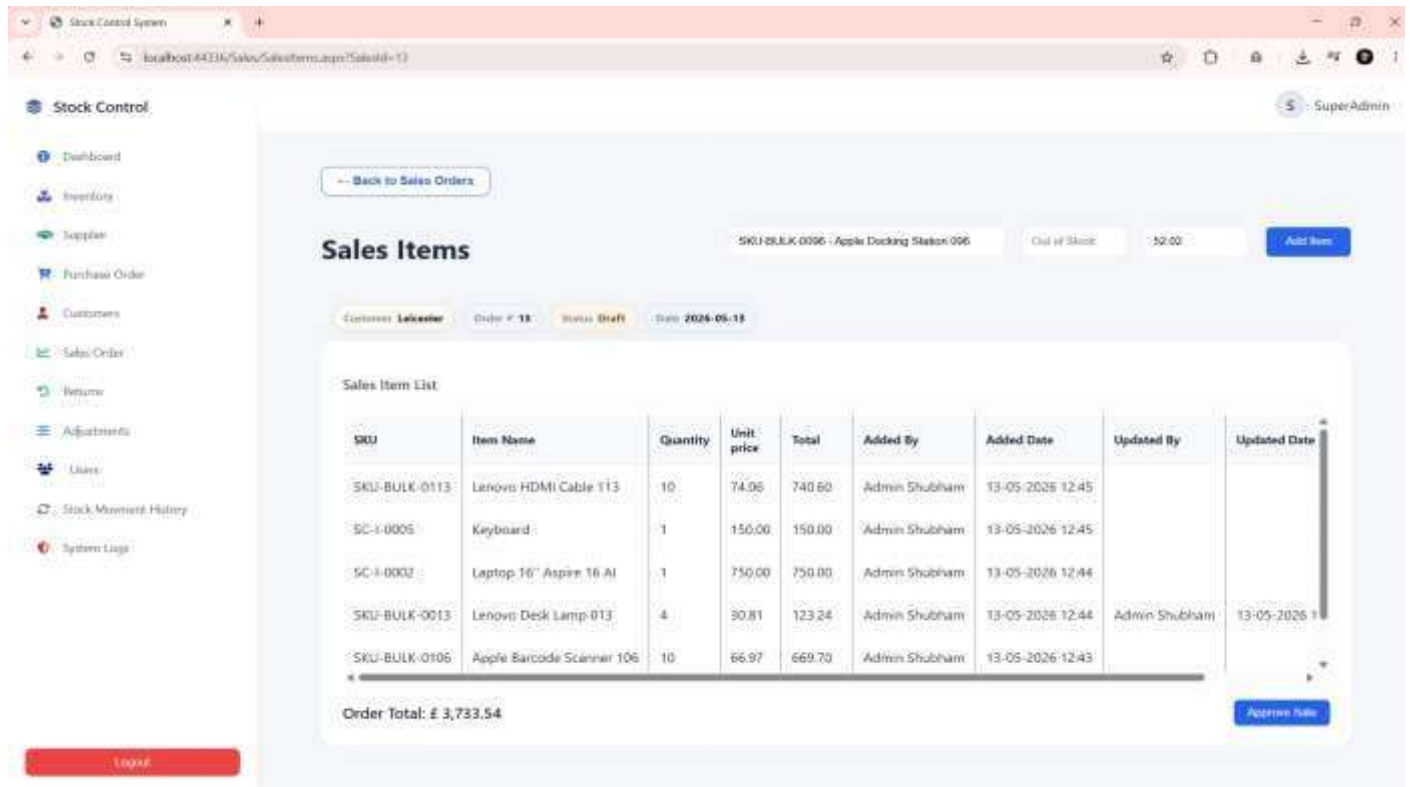


Figure 6.16 – Sales item Validation: Out Of stock Item and Add Item Button Disabled.

Figure 6.16 shows the interface response when an out-of-stock item is selected. The quantity field indicates that the item is out of stock and the Add Item button is disabled. This improves the user experience while supporting the same stock-control rule enforced on the server side.

6.8 Sales Approval and Low-Stock Email Alerts

Sales approval is restricted to Admin and SuperAdmin users. This prevents Staff users from completing the final stock deduction without authorisation. The sales workflow therefore separates data entry from approval: Staff users may prepare sales records where permitted, but the stock-changing approval step is controlled by higher-privilege roles.

When a sale is approved, the system reduces the relevant inventory quantities, records a **SALE / OUT** movement in the StockLedger table and records the approval action in the AuditLogs table. After this, the system checks whether any affected items have fallen below their configured minimum stock level. If low-stock items are found, an email alert is sent to notify administrative users.

The low-stock alert is placed after sales approval because stock levels only become critical once the sale has actually reduced inventory. This avoids sending alerts for draft sales that may never be approved. The email-generation logic is discussed later in **Listing 10.5**, where the system builds an HTML table of low-stock items and marks alerts as sent to avoid repeated emails.

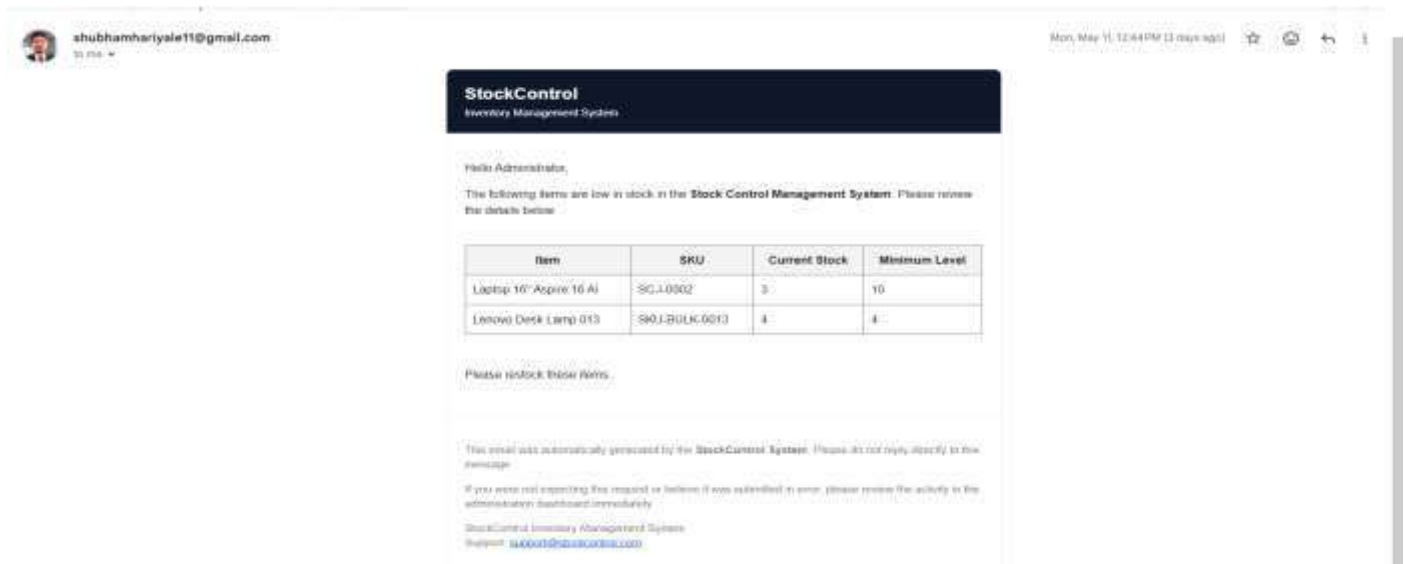


Figure 6.17 – Low Stock Email Alert sent after sales approval

Figure 6.17 shows the low-stock email generated by the system. The email identifies the affected items and displays the item name, SKU, current stock and minimum level. This allows Admin and SuperAdmin users to quickly see which products require restocking without manually checking the inventory page after every sale.

This feature supports the reliability and usability requirements of the project. From a reliability perspective, the system continues monitoring stock after sales activity. From a usability perspective, the user does not need to constantly inspect the inventory table to notice low-stock items. The email alert therefore turns stock monitoring into an automated follow-up action within the workflow.

6.9 Returns Implementation

The returns module supports both **sales returns** and **purchase returns** within a single workflow. This was designed to improve usability while still enforcing stock-control rules. Instead of allowing users to manually type arbitrary order and item details, the page uses **cascading selection logic**. Each selection controls the next one, which reduces invalid combinations and improves data accuracy.

For a **sales return**, the user selects:

Sales Return → Customer → Sales Order → Item → Quantity → Reason

For a **purchase return**, the user selects:

Purchase Return → Supplier → Purchase Order → Item → Quantity → Reason

This approach is stronger than presenting all orders and items in one uncontrolled list. In practice, the selected customer restricts the available sales orders, and the selected sales order restricts the available items. The same principle is used for purchase returns with suppliers and purchase orders. As a result, the user is guided through a valid workflow rather than being expected to remember identifiers manually.

The page also stores the selected record identifiers in hidden fields. This is important because the server validates the actual IDs, not just the visible text shown in the search boxes. The detailed hidden-ID and return quantity validation logic is discussed later in **Listing 10.2**.

The return workflow calculates the remaining allowed quantity before creating the return. For a sales return, the user cannot return more than the quantity originally sold after considering any previous returns. For a purchase return, the system also considers the quantity originally purchased, previous returns and current stock. This avoids situations where the system would incorrectly increase stock or remove more stock than exists.

The final return action is performed through the service workflow layer, which creates the return transaction, updates inventory, writes the corresponding record into the StockLedger table and logs the event in AuditLogs. In this way, return handling is not treated as a simple form submission. It becomes part of the controlled stock-movement process used throughout the application.

The screenshot shows a web application interface for a 'Stock Control System'. A modal dialog titled 'Add New Return' is open in the center. The dialog contains the following fields and information:

- Return Type:** Sales Return (selected from a dropdown)
- Customer:** HMD Register (Mr Wong)
- SO:** SO-2
- Item:** Laptop HP Aquar T641G0-1 (0007)
- Qty:** 1
- Returned:** 1
- Max:** 4

At the bottom of the dialog are two buttons: 'Create Return' (in blue) and 'Cancel' (in grey). The background shows a 'Returns' table with columns for Return ID, Qty, Item, and Status. The table contains several rows of data, including returns for 'Laptop HP Aquar T641G0-1 (0007)' and 'Laptop HP Aquar T641G0-1 (0007)'.

Figure 6.18 – sales return creation form with calculated maximum return quantity

Figure 6.18 shows the sales return form after the user has selected the return type, customer, sales order and item. The form displays the calculated return context, including the quantity sold, quantity already returned and the maximum quantity still allowed. This helps guide the user before submission and reduces input errors.

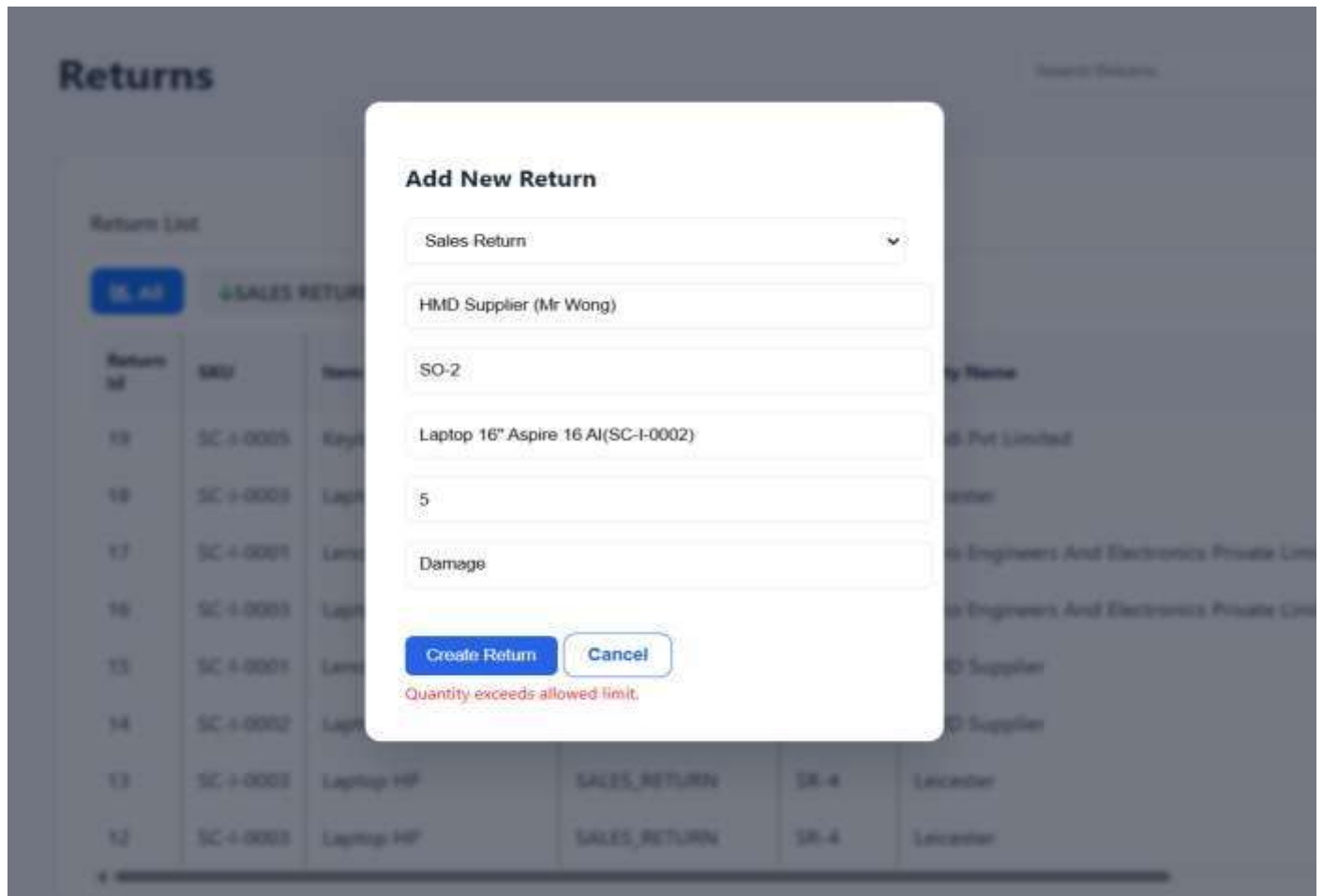


Figure 6.19 – Sales return Validation when quantity exceed the allowed limit

Figure 6.19 shows the validation message displayed when the entered return quantity exceeds the allowed limit. This confirms that the system does not allow a sales return to exceed the remaining valid return quantity. The screenshot provides evidence that return validation is enforced at the application level.

Figure 6.20 – purchase return form with maximum return quantity based on purchase and available stock

Figure 6.20 shows the purchase return workflow. In this example, the form displays the purchased quantity, returned quantity, current stock and calculated maximum return quantity. This is particularly important because a purchase return reduces inventory, so the system must ensure that the return does not exceed either the remaining purchase quantity or the stock currently available.

6.10 Stock Adjustment Implementation

The **stock adjustment module** was implemented to support manual correction of inventory when the physical stock level does not match the recorded system quantity. In a real stock-control environment, this may happen because of damaged items, missing stock, counting errors, misplaced goods or other operational discrepancies. For this reason, the system needs a controlled way to manually increase or decrease stock while still preserving traceability. The adjustment form supports two movement types:

- **IN** – increases stock quantity
- **OUT** – decreases stock quantity

When a user selects an item, the page stores the selected item identifier in a hidden field and displays the current stock level through the quantity placeholder. This improves usability by giving immediate feedback before the user submits the adjustment.

To ensure that the user selected a valid item from autocomplete rather than typing free text only, a custom validator checks that the hidden item ID is present:

```
protected void ValidateItem(object source, ServerValidateEventArgs args)
{
    args.IsValid = !string.IsNullOrEmpty(hfItemId.Value);
}
```

Listing 6.7 – Custom Validation for Selected Adjustment Item

This validation is important because the stock adjustment workflow should operate on a valid inventory record, not just on visible text typed into the input box. If the hidden item identifier is missing, the adjustment request is rejected before any database logic is executed.

Once the form has been validated, the page collects the selected item, quantity, movement type and reason, then passes the operation to the workflow service:

```
int userid = Convert.ToInt32(Session["UserId"]);
int itemId = Convert.ToInt32(hfItemId.Value);
string type = ddlType.SelectedValue; string
reason = txtReason.Text; int qty =
Convert.ToInt32(txtQty.Text);

flow.CreateAdjustmentWithTransaction(itemId, qty, type, reason, userid);
```

Listing 6.8 – Adjustment submission to workflow service

This code is significant because it shows that the page itself does not directly update stock quantities. Instead, it sends the request to the service workflow layer, where the business logic can coordinate the stock change, stock ledger recording and audit logging in a controlled way. This follows the layered architecture used across the system.

A particularly important rule in this module is **negative-stock prevention**. If the adjustment type is **OUT**, the system checks whether sufficient stock exists before allowing the operation. If the entered quantity is greater than the current available stock, the adjustment is rejected, the modal stays open and an error message is displayed. This prevents users from manually pushing stock below zero.

This rule is important because stock adjustments are inherently high-risk operations. Unlike sales and purchases, adjustments are manual corrections, so if they were not properly validated they could easily damage stock integrity. By validating adjustment OUT operations, the module ensures that manual correction remains possible without creating invalid inventory states.

The final adjustment workflow also records the event in the **StockLedger** table and the **AuditLogs** table. This means a stock adjustment does not simply change the quantity silently. Instead, it becomes a traceable stock movement with a recorded reason, user and timestamp. This design is valuable from both an operational and an auditing perspective.

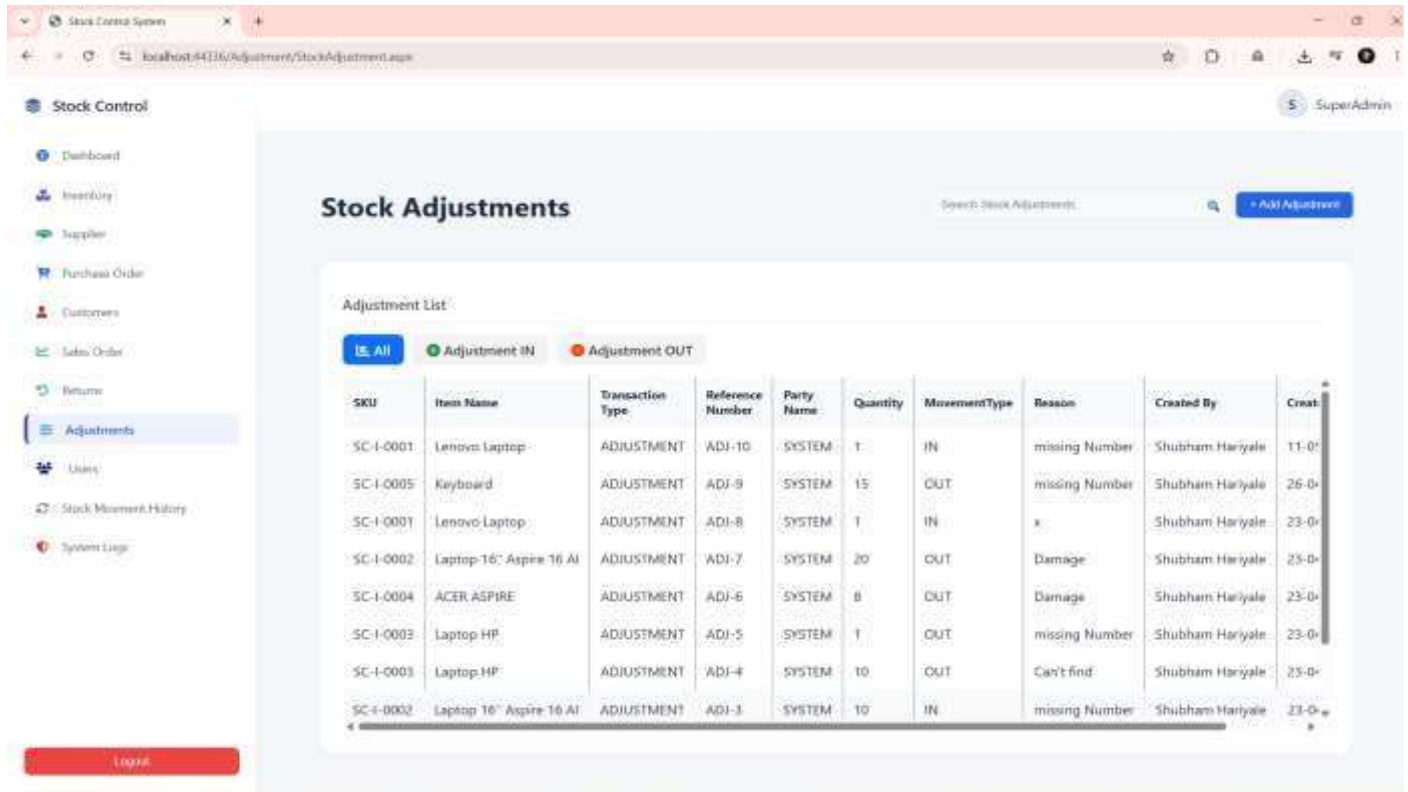


Figure 6.21 – Stock Adjustment page

Figure 6.21 shows the stock adjustment page, which displays adjustment records together with filter tabs for all, adjustment IN and adjustment OUT movements. The page supports traceability by showing the item, transaction type, reference number, movement type, reason, user and date. This provides a historical record of manual stock corrections.

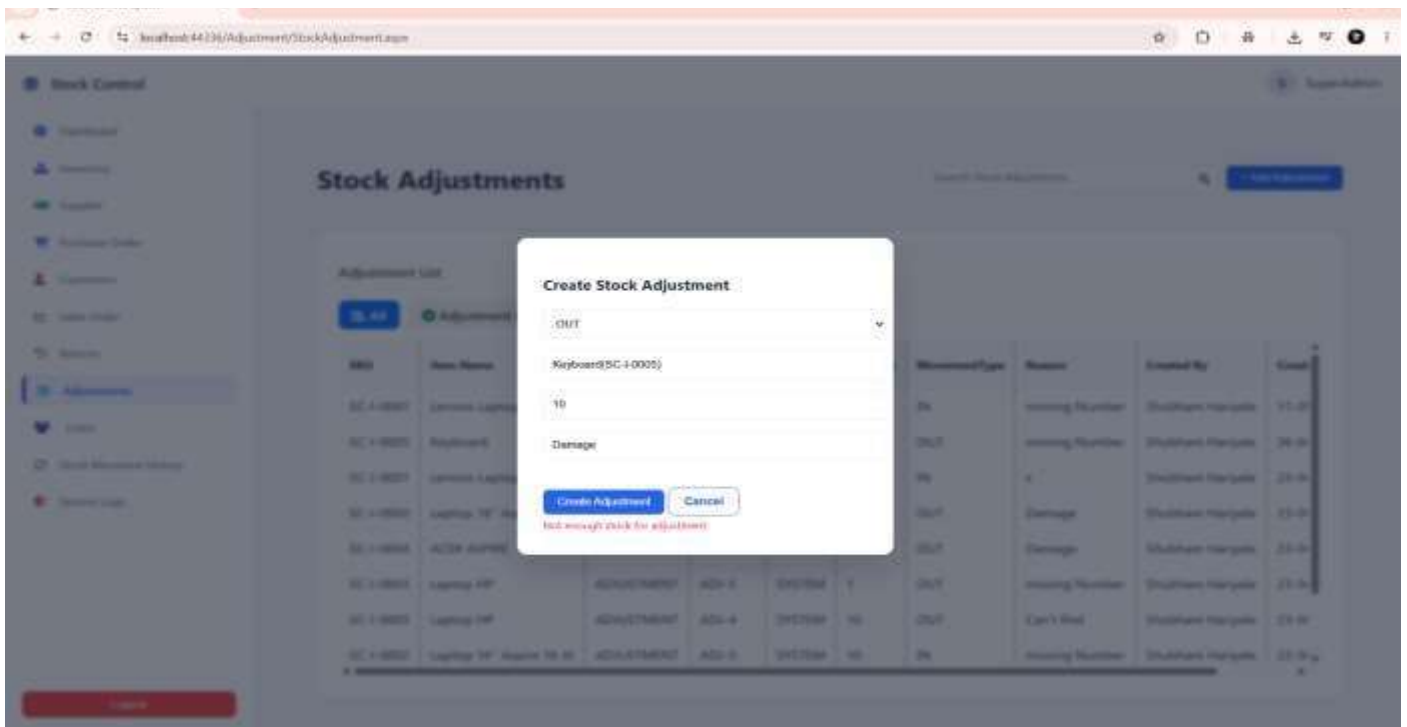


Figure 6.22 – Adjustment Out Negative Stock Prevention

Figure 6.22 shows the stock adjustment modal when the user attempts to perform an adjustment OUT operation that exceeds the available stock. The system displays the message **“Not enough stock for adjustment”** and keeps the modal

open so that the user can correct the input. This demonstrates that the module prevents negative stock during manual adjustment.

6.11 Stock Movement History Implementation

The **Stock Movement History** page displays operational stock movements recorded in the StockLedger table. This page is different from the inventory page because it does not only show the current quantity of an item. Instead, it explains **how and why stock changed** over time.

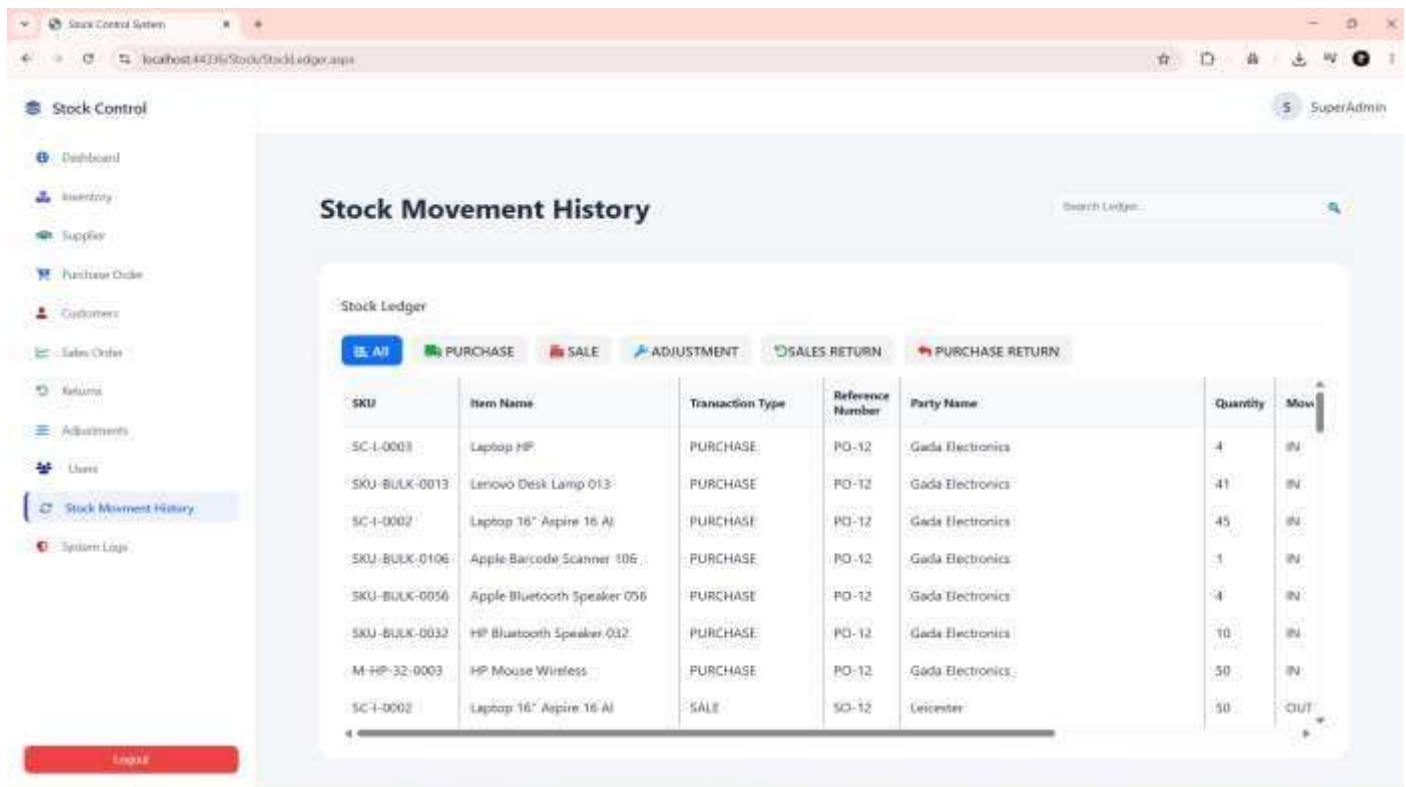
The page includes filters for:

- All
- Purchase
- Sale
- Adjustment
- Sales Return
- Purchase Return

Each stock movement record displays key information such as SKU, item name, transaction type, reference number, party name, quantity, movement type, created by and created date. This makes the page useful for tracing stock activity across the full system.

This feature is important because stock changes should not be invisible. If an item quantity increases, the movement history can show whether the increase came from a purchase approval, sales return or adjustment IN. If an item quantity decreases, the page can show whether the decrease came from a sale, purchase return or adjustment OUT.

The autocomplete search also uses the active filter as context. For example, when the user filters by SALE, the suggestions are based on sale-related movements rather than all ledger records. This improves usability because the search results are more relevant to the selected transaction category.



Stock Movement History

Search Ledger:

Stock Ledger

SKU	Item Name	Transaction Type	Reference Number	Party Name	Quantity	Mov
SC-I-0003	Laptop HP	PURCHASE	PO-12	Gada Electronics	4	IN
SKU-BULK-0013	Lenovo Desk Lamp 013	PURCHASE	PO-12	Gada Electronics	41	IN
SC-I-0002	Laptop 16" Aspire 16 A1	PURCHASE	PO-12	Gada Electronics	45	IN
SKU-BULK-0106	Apple Barcode Scanner 106	PURCHASE	PO-12	Gada Electronics	1	IN
SKU-BULK-0056	Apple Bluetooth Speaker 056	PURCHASE	PO-12	Gada Electronics	4	IN
SKU-BULK-0032	HP Bluetooth Speaker 032	PURCHASE	PO-12	Gada Electronics	10	IN
M-HP-32-0003	HP Mouse Wireless	PURCHASE	PO-12	Gada Electronics	30	IN
SC-I-0002	Laptop 16" Aspire 16 A1	SALE	SO-12	Leicester	30	OUT

Figure 6.23 – Stock Movement history Page showing filtered stock transaction

Figure 6.23 shows the Stock Movement History page. The page provides a unified view of stock movements across purchases, sales, stock adjustments, sales returns and purchase returns. The filters allow users to focus on a specific

movement type, while the table provides evidence of the item, transaction source, reference number, party, quantity and movement direction.

This page supports the traceability objective of the project. Instead of only storing the latest stock quantity, the system keeps a history of stock-changing events. This makes stock changes explainable and supports later investigation if an item quantity appears incorrect.

6.12 System Audit Logs Implementation

The **System Audit Logs** page records wider user and system activity across the application. It is different from Stock Movement History because it does not only record physical stock movement. Instead, it records important actions, page access events, errors and unauthorised access attempts.

The audit page includes filters for:

- All
- Authentication
- Inventory
- User Management
- System Usage
- System Errors
- Unauthorized Access

This structure makes the audit log easier to inspect because Admin, SuperAdmin and Auditor users can filter the log by event category rather than reading every event manually.

Audit logging is centralised through the AuditLoggerService and AuditLoggerDAL classes. Pages do not directly insert audit records into the database. Instead, they call the shared logging service, which passes the event to the data access layer and then to the `sp_AuditLogged_Insert` stored procedure. The detailed audit logging code is discussed later in **Listing 10.4**.

The audit log records more than a simple action message. It captures contextual request information such as page name, IP address, device, browser, event category, event type, severity and timestamp. This supports accountability because the log can show who performed an action, where it happened, what type of event it was and how serious it was.

This is one of the strongest accountability features in the project. For example, if a user tries to create an invalid stock adjustment, the system can record the failed operation as a system error. If a user opens the sales item page, the event can be recorded as system usage. If a restricted action is attempted, it can be recorded under unauthorised access.

Figure 6.25 shows the exported audit log opened in Excel. The exported file includes audit fields such as log ID, user name, email, role, event category, event type, severity, description, page name, IP address, device, browser and timestamp. This demonstrates that audit evidence is not trapped inside the web interface and can be reviewed separately when required.

Together, the Stock Movement History and System Audit Logs pages provide two different but related forms of traceability. The Stock Movement History page explains **what happened to stock**, while the System Audit Logs page explains **what users and the system did**. This distinction is important because a reliable stock-control system needs both operational history and accountability evidence.

7. Testing and Evaluation

7.1 Testing Strategy

The testing strategy was designed to evaluate whether the system meets its main aim: to provide secure, role-based stock control with reliable stock movement and audit traceability. The system was tested across the main operational workflows, including authentication, user access, inventory management, purchases, sales, returns, stock adjustments, stock movement history and system audit logs.

The testing covered:

- Functional testing
- Role-based access testing
- Validation testing
- Negative-stock testing
- Stock movement testing
- Return quantity testing
- Sales price testing
- Audit logging evidence
- CSV and Excel export testing • Usability observation

Testing was mainly manual because of the project timeframe, but it was carried out systematically. Each major workflow was tested using both valid and invalid data. This was important because a stock-control system must not only work when users enter correct data; it must also respond safely when users enter incorrect quantities, invalid selections or attempt actions outside their role.

Screenshots were collected as evidence throughout testing. These screenshots support the testing tables by showing working pages, validation messages, disabled actions, audit records, stock movement history and exported audit logs.

7.2 Functional Testing Results

Test ID	Test Case	Expected Result	Actual Result	Status
FT1	Login with valid account	User reaches dashboard	Dashboard opened	Pass
FT2	Login with invalid account	Access rejected	Error shown	Pass
FT3	Access protected page without login	Redirect/deny access	Access blocked	Pass
FT4	Create customer with missing fields	Validation summary shown	Validation shown	Pass
FT5	Create customer with valid fields	Customer added	Customer appeared in grid	Pass

FT6	Search customer using autocomplete	Suggestions shown	Suggestions displayed	Pass
FT7	Create sales order without customer	Validation rejects form	"Please Select Customer" shown	Pass
FT8	Create sales order with valid customer	Draft sales order created	Redirected to SalesItems page	Pass
FT9	Select out-of-stock item	Add button disabled	Button disabled	Pass
FT10	Add sales item below selling price	Rejected	Error shown	Pass
FT11	Add same item with different price	Rejected	Error shown	Pass
FT12	Approve empty sale	Rejected	Zero-total error shown	Pass
FT13	Create sales return above remaining quantity	Rejected	Quantity limit error shown	Pass
FT14	Create purchase return above available stock	Rejected	Quantity limit enforced	Pass
FT15	Create adjustment OUT above stock	Rejected	"Not enough stock" shown	Pass
FT16	Search stock movement by item	Matching movement records shown	Filtered result shown	Pass
FT17	Search audit logs	Matching audit logs shown	Filtered result shown	Pass
FT18	Export audit logs to CSV	CSV downloads	CSV generated	Pass
FT19	Export audit logs to Excel	Excel downloads	XLS generated	Pass
FT20	Low-stock after sale approval	Email alert sent	Email received	Pass

Table 7.1 – Functional Testing Result

The functional testing results show that the main workflows were implemented successfully. The system supports standard operations such as login, searching, creating customers, creating sales orders and exporting audit logs. More importantly, the tests also confirm that the system rejects unsafe actions, including out-of-stock sales, sales below the configured selling price, excessive return quantities and stock adjustment OUT operations that would create negative stock.

This is important because the project is not only assessed on whether pages open and records can be created. The main technical value of the system is that invalid business operations are blocked before they damage inventory data.

7.3 Role-Based Access Testing

Test ID	Role	Action Tested	Expected Result	Status
RB1	SuperAdmin	View all operational pages	Allowed	Pass
RB2	SuperAdmin	Manage users	Allowed	Pass
RB3	Admin	Approve sale	Allowed	Pass
RB4	Admin	View audit logs	Allowed	Pass
RB5	Staff	Create sales order	Allowed	Pass
RB6	Staff	Approve sales	Button hidden / not allowed	Pass
RB7	Staff	View sensitive audit logs	Not allowed	Pass
RB8	Auditor	View customer list	Read-only view	Pass

RB9	Auditor	Add customer	Button hidden	Pass
RB10	Auditor	Add return	Button hidden	Pass
RB11	Auditor	Add adjustment	Button hidden	Pass
RB12	Auditor	View Stock Movement History	Allowed	Pass
RB13	Auditor	View System Audit Logs	Allowed	Pass

Table 7.2 – Role Based Access Testing

The results show that role restrictions are not only visual. They are also supported by backend page checks using session validation and role validation. This matters because hiding a button is not enough security. A user may still attempt to access a restricted page through a direct URL. The use of backend role checks helps protect restricted workflows even when the interface is bypassed.

The role testing also confirms that the Auditor role works as intended. Auditors can review stock movement and audit evidence, but they cannot create, edit or approve operational records. This supports the accountability aim of the project because audit users are given visibility without being given unnecessary control over stock-changing actions.

7.4 Validation and Negative-Stock Testing

Area	Invalid Input	System Behaviour	Result
Sales item	Quantity greater than available stock	Rejected before insert	Pass
Sales item	Out-of-stock product	Add button disabled	Pass
Sales item	Price below selling price	Rejected	Pass
Sales item	Same item with different price	Rejected	Pass
Sales approval	Zero order total	Rejected	Pass
Sales return	Quantity greater than sold remaining	Rejected	Pass
Purchase return	Quantity greater than current stock	Rejected	Pass
Adjustment OUT	Quantity greater than stock	Rejected	Pass
Customer create	Missing required fields	Validation summary shown	Pass
Item selection	Typed text without autocomplete ID	Custom validator rejects	Pass

Table 7.3 – Validation and Negative-Stock Testing result

The most important result is that negative stock is prevented across more than one workflow. The system does not only check stock on the sales page. It also checks stock during purchase returns and stock adjustments. This is important because negative stock could be created through different routes if validation was only implemented in one module.

The validation also confirms that autocomplete is not treated as decoration only. Where item or party selection matters, the system checks hidden IDs to confirm that the user selected a real database record. This reduces the risk of users typing text that looks valid but is not linked to an actual item, customer or supplier record.

7.5 Stock Movement Testing

Operation	Before	Action	After	Ledger Result
Purchase approval	0	Approve purchase qty 10	10	PURCHASE / IN
Sales approval	10	Approve sale qty 3	7	SALE / OUT
Sales return	7	Return qty 1	8	SALES_RETURN / IN
Purchase return	8	Return qty 2	6	PURCHASE_RETURN / OUT
Adjustment IN	6	Add qty 4	10	ADJUSTMENT / IN
Adjustment OUT	10	Remove qty 3	7	ADJUSTMENT / OUT

Table 7.4 – Stock movement testing

Table 7.4 confirms that each stock-changing workflow updates the inventory quantity in the expected direction and creates a matching movement record. Purchase approval and sales returns increase stock, while sales approval, purchase returns and adjustment OUT reduce stock. Adjustment IN increases stock.

This test is important because it proves that the Stock Movement History page is not just a display page. It is connected to the actual stock-changing workflows. Each change in quantity can be explained by a transaction type and movement direction.

7.6 Audit Log Evaluation

The audit log was tested by performing operations and then checking the System Audit Logs page. Audit evidence was generated for page access, item operations, sales item additions, sale approvals, purchase approvals, return creation, adjustment creation, system errors and unauthorised access.

Evidence Area	Observation	Interpretation
Page access	Page access events recorded	Page-level logging works.
Sales item activity	Sales item additions recorded	Sales workflow is logged.
Purchase approval	Approval events recorded	Purchase approval is traceable.
Sale approval	Approval events recorded	Sale approval is traceable.
Returns	Return creation events recorded	Return workflow is logged.
Stock adjustments	Adjustment events recorded	Manual stock correction is traceable.
Errors	Critical/error events recorded	Debugging and accountability supported.
Export	CSV and Excel export available	Logs can be reviewed externally.

Table 7.5 – Audit Log Evaluation

The audit log evaluation confirms that the system records both normal activity and error-related events. This is important because a reliable stock-control system should not only record successful operations. Failed operations, validation errors and unauthorised actions are also useful evidence, especially when investigating misuse or system problems.

The audit log also complements the Stock Movement History page. The Stock Movement History page explains what happened to inventory, while the System Audit Logs page explains what users and the system did. Together, they provide stronger traceability than either feature would provide alone.

7.7 Security Testing

Test	Expected Behaviour	Result
Access protected page without login	Redirect or deny access	Pass
Direct URL to restricted page	Backend role check applies	Pass
Staff attempts approval action	Button hidden or action blocked	Pass
Auditor attempts creation action	Button hidden or action blocked	Pass
Search with special characters	Treated as text	Pass
SQL input through search boxes	Passed as parameter	Pass

Table 7.6 – Security Testing

The security testing confirms that authentication and role validation are applied beyond the visible interface. The system blocks protected pages when a valid session is not present and applies role checks to restrict sensitive pages and actions.

The system also uses stored procedures and parameterised SQL commands. This reduces SQL injection risk compared with string-concatenated SQL because user input is passed as parameter values rather than being directly inserted into SQL statements. Although this testing was manual rather than a full penetration test, it provides evidence that the main security controls are present and working.

7.8 Performance and Scalability Observation

Although full load testing was outside the scope of the project, basic performance observations were carried out during manual testing. The system was tested with growing numbers of inventory, sales, return, stock movement and audit log records. Search and filtering remained usable because the application does not manually load unrelated data into the interface. Instead, search terms and filter types are passed to stored procedures, which return matching records.

The most performance-sensitive pages are **Stock Movement History** and **System Audit Logs** because these tables can grow continuously over time. To reduce this issue, both pages include filtering and autocomplete search. The audit log page also supports CSV and Excel export so that larger reviews can be carried out outside the web interface.

A future production version should include formal load testing with a larger dataset, indexes on common search fields and pagination for very large tables. However, the implemented approach demonstrates scalability awareness because the design already avoids unrestricted manual browsing through all records where filtering is required.

7.9 Usability Evaluation

The interface was tested manually across the major roles. Several usability strengths were identified:

Usability Feature	Benefit
Sidebar navigation	Makes modules easy to access.
Role-based visibility	Reduces clutter and hides unavailable actions.
Modal forms	Reduce unnecessary page navigation.
Autocomplete	Reduces invalid selections and speeds up searching.
Cascading autocomplete in returns	Makes the return workflow clearer and safer.
Quantity placeholders	Show useful context such as available stock or maximum return quantity.
Filter tabs	Make returns, adjustments, stock movement and audit logs easier to review.
Validation messages	Prevent invalid records before database changes.

Table 7.7 – Usability strengths identified during testing

Some limitations were also identified:

Limitation	Suggested Improvement
Some validation messages alert use browser summaries.	Replace with more consistent inline messages.
Some headings required naming cleanup.	Use final terms consistently, such as “Stock Movement History” and “System Audit Logs”.
Some tables require horizontal scrolling.	Add responsive column priorities or export/report views.

Table 7.8 – Usability Limitation and improvements

These issues do not prevent the system from functioning, but correcting them would improve the professional feel of the interface. The naming cleanup is especially important because terminology affects how clearly users understand the system. During development, terms such as “Stock Ledger” and “Login Audit History” were revised to “Stock Movement History” and “System Audit Logs” because these names better describe what the pages actually display.

7.10 Critical Evaluation of Testing

A key strength of the project is that the system does not treat inventory as a single editable number. Instead, most stock changes are created from business events such as purchase approval, sales approval, returns or adjustments. This gives the system stronger traceability than a simple inventory table because the current quantity can be explained through stock movement records.

The testing also shows that validation is applied across several workflows rather than only in one place. Sales, returns and adjustments all include checks designed to prevent invalid stock movement. This is important because stock integrity can be broken through any workflow that changes quantity.

However, one limitation is that testing was mainly manual. Manual testing was suitable for confirming the main workflows within the project timeframe, but a production system would benefit from automated regression tests, database transaction tests and more formal concurrent-user testing. This is particularly important for sales approval and stock adjustment workflows, where multiple users could potentially work with the same item at similar times.

Another limitation is that the system is still mainly operational rather than predictive. It can warn users when stock becomes low, but it does not yet forecast future demand. During review discussions, the possibility of using machine learning for stock forecasting was raised. This was not implemented because the current project time was focused on building a reliable workflow-based system first. In future work, the stock movement history could provide useful longitudinal data for forecasting demand, identifying seasonal patterns and recommending reorder quantities.

Overall, the testing results show that the system meets the main functional and non-functional requirements. It supports secure access, controlled stock movement, validation, audit logging and exportable evidence. The main areas for improvement are not basic functionality, but deeper automation, formal load testing, stronger concurrency testing and more polished usability refinements.

8. Project Management and Development Process

8.1 Development Approach

The project followed an incremental development approach. This was suitable because the system depended on several connected workflows, and it would have been risky to build everything at once without first proving the foundation.

The first stage focused on the database schema, authentication, role model and inventory records. These parts were needed before the operational workflows could be implemented safely. After this, the supplier and customer modules were added because they provide the master data required for purchase and sales workflows. The purchase and sales modules were then developed, followed by returns, stock adjustments, stock movement history, audit logging, email alerts and final testing.

This approach reduced risk because each stage produced a working part of the system before the next workflow was added. It also allowed the same layered pattern to be reused across the project: ASPX page, code-behind, service class, data access layer class and SQL Server stored procedure. Reusing this pattern made the project easier to extend and helped keep the implementation consistent.

A major benefit of the incremental approach was that it allowed the project to evolve from a simple inventory management system into a more realistic stock-control system. Instead of only storing product quantities, the system gradually developed into a workflow-based application where stock changes are approved, validated, recorded and auditable.

8.2 Project Phases

Phase	Main Work	Outcome
Phase 1	Database foundation, login and roles	Secure base system created.
Phase 2	Inventory, suppliers and customers	Master data modules completed.

Phase 3	Purchase orders and purchase items	Stock IN workflow implemented.
Phase 4	Sales orders and sales items	Stock reservation and approval workflow implemented.
Phase 5	Returns	Sales return and purchase return workflows implemented.
Phase 6	Stock adjustments	Manual correction workflow implemented.
Phase 7	Stock movement history	Unified stock movement reporting implemented.
Phase 8	Audit logs and export	Accountability and evidence export implemented.
Phase 9	Email alerts	Low-stock notification implemented.
Phase 10	Testing and report writing	Evidence collected and evaluated.

Table 8.1 – Project Phases

The phases show that the project was not developed as disconnected pages. Each phase depended on the previous one. For example, purchases and sales required suppliers, customers and inventory items to exist first. Returns required approved sales or purchases to exist before valid return quantities could be calculated. Stock Movement History and System Audit Logs were added after the main workflows so that the system could record and explain the actions generated by those workflows.

8.3 Risk Management

Risk	Impact	Mitigation
Scope expansion	High	Focused on stock-control workflows instead of full ERP/accounting.
Negative stock bugs	High	Added validation in sales, returns and adjustments.
Role-permission mistakes	High	Tested each role separately and hid restricted actions.
Transaction complexity	High	Used service workflows and stored procedures.
Web Forms complexity	Medium	Used layered architecture rather than placing everything in code-behind.
Time pressure	High	Reused design patterns across similar modules.
Testing limitations	Medium	Combined manual test tables, screenshots and audit evidence.
UI inconsistencies	Low/Medium	Identified final naming corrections for future polish.

Table 8.1 – Risk Management

The highest-risk areas were stock integrity and access control. A stock-control system becomes unreliable if users can create negative stock, approve unauthorised transactions or change quantities without explanation. For this reason, the project focused strongly on validation, role restrictions and traceability.

Some risks were reduced through design decisions rather than only testing. For example, stock is not changed when a draft sales or purchase order is created. It changes only through approval or controlled stock workflows. Similarly, audit logging and stock movement recording were added to make important actions visible after they happen.

8.4 Changes During Development

The project originally began as a stock-control and inventory management system. At the start, the main focus was on storing products, suppliers, customers and quantities. During development, it became clear that simple inventory CRUD would not be enough to demonstrate strong technical achievement or reflect a realistic stock-control process.

The project therefore evolved into a workflow-based stock-control system. This introduced purchase approval, sales approval, return handling, stock adjustments, stock movement history, audit logging and low-stock email alerts. This change increased the complexity of the project, but it also made the final system much stronger because stock changes became controlled and explainable.

One important design change was the decision to focus on **stock movement** rather than only **stock quantity**. In a simple inventory system, the current quantity might be updated directly. In this project, stock changes are linked to business events such as purchase approval, sale approval, return creation and adjustment. This makes the system more useful for investigation because the user can see why a quantity changed, not just what the current number is.

This change improved the relationship between requirements, design, implementation and testing. The final system is not just a collection of forms. It is a connected workflow where user roles, business rules, database operations, stock history and audit logging work together.

8.5 Response to Review Feedback

Feedback from earlier assessment stages influenced the final direction of the project. The preliminary feedback indicated that the project needed a clearer connection between tools, techniques, aims and objectives. In response, the final implementation focused more strongly on showing how each technical decision supported the project aim.

For example, role-based access control was not treated only as a login feature. It became a design principle applied across page access, sidebar navigation, button visibility, approval permissions and auditor restrictions. Similarly, stored procedures were not included only as a database technique. They were used to support safer, centralised database operations for stock-changing workflows.

The review process also highlighted the need for a stronger evaluation approach. As a result, the final testing strategy was expanded to include functional workflow testing, role-based access testing, validation testing, negative-stock testing, stock movement testing, audit log evidence, export testing and usability observation. This made the evaluation more systematic and better connected to the original aim of building a secure and reliable stock-control system.

The feedback also encouraged consideration of future technical extensions, particularly the possibility of using machine learning for stock forecasting. This was not implemented in the current version because the priority was to build a reliable workflow-based system first. However, it has been included as future work because the Stock Movement History data could provide useful historical stock movement records for forecasting demand, identifying seasonal patterns and recommending reorder quantities in a future version.

9. Critical Evaluation

9.1 Strengths

The main strength of the system is its controlled stock movement design. Instead of allowing users to freely edit current quantities without explanation, the system routes stock changes through defined workflows: purchases, sales, returns and stock adjustments. This creates a stronger audit trail and reduces the risk of unexplained stock changes. It also makes the system more realistic because stock changes are linked to business events rather than being treated as isolated number updates.

A second strength is the role-based access model. The system separates responsibilities between SuperAdmin, Admin, Staff and Auditor. This is suitable for a stock-control environment because not every user should have permission to approve sales, approve purchases, adjust stock or manage users. The Auditor role is particularly useful because it allows review access without giving the user permission to change operational data.

A third strength is the sales workflow. This is one of the most technically developed parts of the system. It prevents out-of-stock items from being added, checks available quantity, considers quantities already added to the same draft sale, blocks sale prices below the configured selling price, prevents the same item being added with different prices in one order and requires approval before final stock movement. These rules reduce the chance of overselling, underselling and inconsistent order records.

A fourth strength is the returns workflow. The cascading autocomplete design guides the user through a valid sequence of selections instead of allowing arbitrary item and order combinations. For example, a sales return requires the user to select the return type, customer, sales order and item before entering quantity and reason. The system then validates the remaining return quantity. This prevents users from returning more than was originally sold or purchased and helps protect stock accuracy.

A fifth strength is accountability. The system separates two types of traceability: stock movement traceability and audit traceability. The Stock Movement History page explains how stock quantities changed, while the System Audit Logs page records user and system activity such as page access, errors, approvals and unauthorised access attempts. The audit log records contextual information including page, device, browser, IP address, severity and timestamp. Together, these features make the system more explainable and easier to investigate.

Another strength is that the project demonstrates a consistent layered implementation pattern. Pages handle interaction, code-behind files manage events, service classes apply business logic, DAL classes call the database and stored procedures perform database operations. This structure improved maintainability because important stock logic was not scattered randomly across individual ASPX pages.

9.2 Limitations

The system also has limitations. First, it is built using ASP.NET Web Forms. This was suitable for the project because it supports server-side controls, validators and event-driven development, but it is not the most modern architecture for a production system. A future version could use ASP.NET Core with a REST API and a modern frontend framework to improve maintainability, testing and scalability.

Second, the project relies mainly on manual testing. The tests were systematic and covered the major workflows, but automated testing would provide stronger repeatable evidence. In particular, automated regression tests would be useful for sales validation, return validation, role-based access and stock adjustment rules.

Third, concurrency testing was limited. The system includes validation and database-level safeguards, but a production system should test simultaneous users attempting to reserve, approve or adjust the same stock item. This would help confirm that stock consistency is preserved under real multi-user pressure.

Fourth, the email alert feature is useful but basic. It proves that the system can automatically notify administrative users when stock becomes low, but a production version would need alert preferences, notification history, retry handling, configurable thresholds and possibly scheduled summary notifications.

Fifth, the system does not include financial stock valuation, invoicing, payments or profit reporting. This is acceptable because the project focuses on stock control rather than accounting. However, it means the system should not be described as a full ERP or financial management platform.

A final limitation is that some user-interface areas would benefit from further polish. The core workflows function correctly, but future improvements could include pagination, better responsive table layouts, inline validation instead of browser alert summaries and consistent final terminology across all headings.

9.3 Evaluation Against Research Question

The research question asked:

How can a secure role-based inventory management system be designed to maintain transactional integrity, stock consistency and audit traceability in a multi-user web environment?

The project answers this question through a combination of design and implementation decisions

Requirement Area	How the Project Responded
Secure access	Authentication, session validation and role-based page restrictions were implemented.
Role separation	SuperAdmin, Admin, Staff and Auditor roles were used to separate responsibilities.
Stock consistency	Stock-changing workflows were controlled through purchases, sales, returns and adjustments.
Transactional integrity	Important stock operations were centralised through service workflow classes and stored procedures.
Validation	Quantity, price and selection validation prevented invalid stock operations.
Traceability	Stock Movement History recorded operational stock movement.
Accountability	System Audit Logs recorded user actions, system events, errors and unauthorised access.
Monitoring	Low-stock email alerts notified administrative users when stock dropped below minimum levels.

Table 9.1 – Evolution against the research question

The project shows that stock consistency cannot be achieved by one feature alone. It requires access control, validation, workflow design, database rules and traceability working together. For example, preventing overselling is not only a sales-page problem. It also depends on stock availability checks, draft-order quantity checks, approval workflow control, database updates and stock movement records.

The implemented system therefore provides a practical answer to the research question. It demonstrates that a secure stock-control system can maintain consistency and traceability by combining role-based access control with controlled stock workflows and audit evidence. Although there are limitations, especially around automated testing and concurrency testing, the completed system meets the main aim of creating a secure, traceable and workflow-driven inventory management application.

10. Technical Achievement

10.1 Technical Achievement Summary

Achievement	Evidence	Why It Matters
Hierarchical RBAC	Four roles with page checks and UI visibility	Protects sensitive actions.
Layered architecture	ASPX, Services, DAL and stored procedures	Improves maintainability.
Controlled stock movement	Purchases, sales, returns and adjustments	Prevents unexplained quantity changes.

Sales reservation logic	Draft sales items reduce available stock	Reduces overselling risk.
Selling-price validation	Sales below selling price rejected	Enforces business rules.
Duplicate-price rule	Same item cannot use different prices in one sale	Prevents inconsistent order data.
Return validation	Remaining quantity calculated	Prevents invalid returns.
Adjustment validation	OUT cannot exceed stock	Prevents negative stock.
Unified stock history	StockLedger and Stock Movement History	Provides operational traceability.
Audit logging	System Audit Logs with export	Supports accountability.
Low-stock alerts	Email sent to Admin/SuperAdmin	Supports stock monitoring.
Autocomplete workflows	Customers, suppliers, orders, items and logs	Improves usability and reduces invalid input.

Table 10.1 – Technical Achievement summary

10.2 Code Extract 1: Sales Item Validation and Reservation

The sales item workflow is one of the most important technical areas of the project because it protects stock before a sale is approved. When a product is added to a draft sales order, the system calculates the quantity already added to that order and compares the final requested quantity with the available stock.

```
int qty = Convert.ToInt32(txtQty.Text);

int userid = Convert.ToInt32(Session["UserId"]); int
salesId = Convert.ToInt32(ViewState["SalesId"]); int
itemId = Convert.ToInt32(hfItemId.Value);

int availableStock = sales.GetAvailableStock(itemId, salesId); int
existingQty = sales.GetExistingItemQuantity(itemId, salesId);

int finalQty = existingQty + qty;

if (finalQty > availableStock)
{
    lblMessage.Text = "Quantity exceeds available stock. Available Stock is "
+ availableStock.ToString();

    lblMessage.Visible = true;
    return;
}
```

Listing 10.1 – sales quantity validation and draft reservation logic

This code is significant because it does not only check the inventory quantity. It also checks the quantity of the same item already added to the current draft sales order. This prevents the user from repeatedly adding the same item until the order exceeds available stock.

For example, if five units are available and the draft order already contains two units of the item, the system only allows three additional units to be added. This supports a reservation-style workflow, where draft sales items are considered before final approval.

10.3 Code Extract 2: Return Quantity Validation

Returns are risky because they can incorrectly increase or decrease stock if users are allowed to return more items than were originally sold or purchased. The return workflow therefore calculates the remaining allowed return quantity before creating the return.

```
int remaining = service.GetRemainingQty(itemId, salesId, purchaseId, type);

if (qty > remaining)
{
    lblMessage.Text = "Quantity exceeds allowed limit.";
    lblMessage.Visible = true;
    lblMessage.ForeColor = System.Drawing.Color.Red;
    modalAddreturn.Show(); return;
}

serviceFlow.CreateReturnWithTransaction(
salesId, purchaseId, itemId, qty,
type, userId,
```

```
reason
);
```

Listing 10.2 – return quantity validation before stock movement

This code demonstrates business-rule validation before a stock-changing transaction is created. For sales returns, the remaining quantity is based on the original sold quantity minus any previous returns. For purchase returns, the logic also protects current stock because a purchase return decreases inventory.

This is important because returns affect stock immediately. A sales return increases stock, while a purchase return decreases stock. Without this validation, users could create false stock increases or remove stock that no longer exists.

10.4 Code Extract 3: Purchase Approval Stored Procedure

The purchase approval stored procedure is one of the strongest database-level examples in the project. It shows how stock-changing logic is protected inside SQL Server rather than being handled only by the page interface. This is important because purchase approval does more than change the purchase status. It must validate the order, increase inventory quantity and create stock movement history.

```
ALTER PROCEDURE [dbo].[sp_ApprovePurchase]
(
    @PurchaseId INT,
```

```
@UserId INT
)
AS
BEGIN
    SET NOCOUNT ON;

    BEGIN TRY
        BEGIN TRANSACTION;

        IF NOT EXISTS
        (
            SELECT 1
            FROM PurchaseOrders WITH (UPDLOCK, HOLDLOCK)
            WHERE PurchaseId = @PurchaseId
        )
        BEGIN
            ROLLBACK TRANSACTION;
            RAISERROR('Purchase order not found.', 16, 1);
            RETURN;
        END

        IF NOT EXISTS
        (
            SELECT 1
            FROM PurchaseOrderItems
            WHERE PurchaseId = @PurchaseId
        )
        OR
        (
            SELECT ISNULL(SUM(TotalPrice), 0)
            FROM PurchaseOrderItems
```

```
WHERE PurchaseId = @PurchaseId
) <= 0
BEGIN
    ROLLBACK TRANSACTION;
    RAISERROR('Cannot approve a purchase with zero total.', 16, 1);
RETURN;
END

IF EXISTS
(
    SELECT 1
    FROM PurchaseOrders
    WHERE PurchaseId = @PurchaseId
    AND Status = 'Approved'
)
BEGIN
    ROLLBACK TRANSACTION;
    RAISERROR('Purchase already approved', 16, 1);
RETURN;
END

UPDATE PurchaseOrders
SET Status = 'Approved',
    UpdatedBy = @UserId,
    UpdatedDate = GETDATE()
WHERE PurchaseId = @PurchaseId;

UPDATE I
SET I.CurrentQuantity = I.CurrentQuantity + POI.Quantity
FROM InventoryItems I
JOIN PurchaseOrderItems POI ON I.ItemId = POI.ItemId
WHERE POI.PurchaseId = @PurchaseId;

INSERT INTO StockLedger
(
    ItemId,
    TransactionType,
    ReferenceId,
    Quantity,
    MovementType,
    CreatedBy,
    CreatedDate
)
SELECT
    ItemId,
    'PURCHASE',
    @PurchaseId,
    Quantity,
    'IN',
    @UserId,
```

```
GETDATE()  
FROM PurchaseOrderItems  
WHERE Purchaseld = @Purchaseld;
```

```
COMMIT TRANSACTION;  
END TRY  
BEGIN CATCH  
    IF @@TRANCOUNT > 0  
        ROLLBACK TRANSACTION;  
  
    DECLARE @Msg NVARCHAR(4000) = ERROR_MESSAGE();  
    RAISERROR(@Msg, 16, 1);  
END CATCH  
END
```

Listing 10.3 - Purchase Approval Stored Procedure with Transaction Handling and Stock Ledger Recording

This stored procedure demonstrates several important technical decisions. First, the procedure uses an explicit transaction so that purchase approval, inventory update and stock ledger insertion are handled as one unit of work. If any part fails, the transaction is rolled back. This supports the ACID principle of atomicity because the system should not approve a purchase without also updating inventory and recording the movement.

Second, the procedure validates the purchase before approving it. It checks that the purchase order exists, that it contains purchase items and that the total value is greater than zero. This prevents empty or invalid purchase orders from increasing stock.

Third, the procedure checks whether the purchase has already been approved. This prevents the same purchase order from being approved twice, which would incorrectly increase stock quantity again.

Fourth, the InventoryItems table is updated by adding the purchased quantities to the current stock. The procedure then inserts a matching PURCHASE / IN record into the StockLedger table. This means the stock increase is not invisible. The system can later show why the inventory quantity changed.

The use of UPDLOCK and HOLDLOCK on the purchase order check also shows concurrency awareness. It helps protect the approval process by locking the relevant purchase order row while the transaction is running, reducing the risk of two approval operations acting on the same purchase order at the same time.

Overall, this stored procedure is a key example of the project's technical achievement because it combines validation, transaction handling, stock update and traceability in one controlled database operation.

10.5 Code Extract 4: Audit Logging with Request Context

The audit logging feature records accountability evidence for user and system activity. It does not only store a simple action message. It also records request context such as page, IP address, browser, device, event category and severity.

```
string ip = SecurityHelper.GetIPAddress(); string
device = SecurityHelper.GetDevice(); string
browser = SecurityHelper.GetBrowser();
string page = HttpContext.Current.Request.Url.AbsolutePath;

using (var conn = db.GetConnection())
using (var cmd = new SqlCommand("sp_AuditLogged_Insert", conn))
{
    cmd.CommandType = CommandType.StoredProcedure;

    cmd.Parameters.AddWithValue("@UserId",          userId);
    cmd.Parameters.AddWithValue("@IP", ip);

    cmd.Parameters.AddWithValue("@Description", description);
    cmd.Parameters.AddWithValue("@Page", page);    cmd.Parameters.AddWithValue("@Device",
device);    cmd.Parameters.AddWithValue("@Browser", browser);
    cmd.Parameters.AddWithValue("@EventType", EventType);
    cmd.Parameters.AddWithValue("@eventcategory", eventcategory);
    cmd.Parameters.AddWithValue("@severity", severity);

    conn.Open();
    cmd.ExecuteNonQuery();
}
```

Listing 10.4 – Audit Logging with Page, Device, Browser and Severity Context

This code is technically important because it centralises audit logging and captures meaningful context. The log can show who performed an action, where it happened, what device and browser were used, and how serious the event was. This supports the Auditor role because auditors need evidence without being able to change operational data.

10.6 Code Extract 5: Low-Stock Email Alert

The low-stock email feature checks for items below their minimum stock level and sends an HTML email notification. This makes stock monitoring more active because users do not need to manually inspect the inventory page after every sale.

```
public void CheckLowStockAndSendAlert()
{
    DataTable dt = dal.CheckLowStockAndSendAlertDAL();

    if (dt.Rows.Count == 0)
    return;

    string rows = "";

    foreach (DataRow row in dt.Rows)
    {
        rows += $"<tr>
            <td>{row["ItemName"]}</td>
            <td>{row["SKU"]}</td>
            <td>{row["CurrentQuantity"]}</td>
            <td>{row["MinimumLevel"]}</td>
        </tr>";
    }

    emailsservice.SendLowStockEmail(rows);

    foreach (DataRow row in dt.Rows)
    {
        dal.MarkLowStockAlertSentDAL(Convert.ToInt32(row["ItemId"]));
    }
}
```

Listing 10.5 – Low-Stock Email Alert Generation

This feature improves the system because low stock becomes an automatic notification rather than a manual checking task. The method builds an email table containing item name, SKU, current stock and minimum level. It then marks the alert as sent, which helps prevent repeated emails for the same low-stock item.

10.7 Autocomplete with hidden ID validation

Autocomplete was used across the system to improve usability and reduce invalid selections. However, one technical issue with autocomplete is that a user can type text that looks valid without selecting a real database record. To avoid this, the system does not trust the visible textbox text alone.

When a user selects a customer, supplier, order or item, the database ID is stored in a hidden field. Custom validators then check that the hidden ID exists before the form is submitted. This approach is used in workflows such as sales item selection, returns and stock adjustments.

```
protected void ValidateItem(object source, ServerValidateEventArgs args)
{
    args.IsValid = !string.IsNullOrEmpty(hfItemId.Value);
}
```


Listing 10.6 – Hidden ID Validation for Autocomplete Selection

This validation is small, but it is important. Product names, customer names or supplier names may be duplicated, mistyped or changed. The database ID gives the system a reliable reference and prevents records being saved against uncontrolled text.

11. Conclusion and Future Work

11.1 Conclusion

This project successfully designed, implemented and evaluated a secure web-based **Stock Control System** using ASP.NET Web Forms, C# and SQL Server. The completed system includes authentication, role-based access control, access requests, user management, inventory management, supplier management, customer management, purchase orders, sales orders, returns, stock adjustments, stock movement history, low-stock email alerts and system audit logs.

The project meets its main aim because it protects stock integrity in a multi-user environment. Stock quantities are not changed randomly or directly without explanation. Instead, they are changed through controlled workflows that validate business rules and record movement history. The system prevents important error cases such as out-ofstock sales, sales below the configured selling price, inconsistent sales item pricing, zero-total sale approval, excessive returns and stock adjustments that would create negative stock.

The project also meets its security and accountability goals. Role-based access control restricts sensitive actions according to user responsibility. Audit logging records important user and system events with detailed context such as page name, browser, device, IP address, severity and timestamp. Stock Movement History records the operational reason for each stock change. Together, these features make the system more trustworthy than a simple inventory CRUD application.

A key achievement of the project is the integration of features. The sales workflow, returns workflow, stock adjustment workflow, stock movement history, audit logs and low-stock alerts are connected. This means the system behaves like a controlled stock-management application rather than a set of separate unrelated pages.

The project does not attempt to become a full enterprise resource planning system. It does not include payment processing, invoicing, financial accounting, stock valuation, barcode scanner hardware integration, delivery management or customer relationship management. These areas were outside the project scope because the main focus was secure stock movement, role-based access control, traceability and reliable workflow design.

Overall, the project demonstrates that a conventional technology stack can still support a robust and professionally usable stock-control system when security, validation, database design and traceability are treated as core design concerns rather than optional additions.

11.2 Future Work

Future development could improve the system in several practical ways.

One important improvement would be **pagination and lazy loading** for large data tables. Pages such as Stock Movement History and System Audit Logs can grow continuously over time. At the moment, the system supports filtering and autocomplete search, but a production version should also load records in smaller pages rather than displaying large result sets at once. This would improve performance, reduce page load time and make the system easier to use with thousands of records.

A second improvement would be stronger **dashboard analytics**. The system already records useful data through sales, purchases, returns, stock adjustments and stock movement history. Future dashboards could show fastmoving items, slow-moving items, low-stock trends, frequent returns, repeated adjustment items and supplier activity. Graphs and trend charts would help Admin and SuperAdmin users understand stock behaviour quickly instead of reviewing only table data.

A third improvement would be **machine learning-based stock forecasting**. This was discussed during the review process as a possible extension. The current system already stores historical stock movement data, which could later

be used to predict demand patterns. For example, a forecasting model could estimate which items are likely to fall below minimum level based on previous sales, seasonal demand, return frequency and supplier lead time. This was not implemented in the current version because the priority was to build a reliable workflow-based stock-control system first.

A fourth improvement would be a more detailed **user profile and activity history**. At present, the system records user actions in audit logs, but a future version could provide a dedicated profile page showing each user's role, account status, recent activity, approval history and security events. This would make user administration clearer and would support audit review.

A fifth improvement would be **barcode scanner integration**. The current system stores barcode-related product information, but it does not yet integrate with scanner hardware. Future work could allow users to scan items during purchase receiving, sales item selection, returns and stock adjustments. This would reduce manual typing and make the system more suitable for real warehouse use.

Another practical extension would be **multi-location stock support**. The current system assumes one main stock location. In a larger organisation, stock may be stored across multiple warehouses, rooms or departments. Future development could add stock locations and allow movement between locations, making the system suitable for more complex operations.

The system could also be extended with **invoicing, payments and stock valuation**, but these should be treated as separate business areas. The current project focuses on stock control rather than accounting. A future version could generate purchase invoices, sales invoices, payment status, profit summaries and stock valuation reports if the project were expanded toward a broader business-management system.

Finally, the user interface could be improved with more polished inline validation, responsive table layouts, daterange filters and export options for additional modules. These changes would not alter the core stock-control logic, but they would make the system more comfortable and professional for everyday use.

Overall, the strongest future direction is to build on the data already being collected. The Stock Movement History and Audit Logs tables provide a useful foundation for analytics, forecasting, compliance review and operational decision-making.

References

- [1] R. S. Sandhu, E. J. Coyne, H. L. Feinstein and C. E. Youman, "Role-Based Access Control Models," *Computer*, vol. 29, no. 2, pp. 38–47, Feb. 1996.
- [2] A. Silberschatz, H. F. Korth and S. Sudarshan, *Database System Concepts*, 7th ed. New York, NY, USA: McGraw-Hill Education, 2019.
- [3] I. Sommerville, *Software Engineering*, 10th ed. Harlow, UK: Pearson, 2016.
- [4] Microsoft, "ASP.NET Web Forms," Microsoft Learn. [Online]. Available: <https://learn.microsoft.com/en-us/aspnet/web-forms/>. Accessed: May 15, 2026.
- [5] Microsoft, "ASP.NET Web Forms Guidance," Microsoft Learn. [Online]. Available: <https://learn.microsoft.com/en-us/aspnet/web-forms/overview/>. Accessed: May 15, 2026.
- [6] Microsoft, "ADO.NET Overview," Microsoft Learn. [Online]. Available: <https://learn.microsoft.com/en-us/dotnet/framework/data/adonet/ado-net-overview>. Accessed: May 15, 2026.

- [7] Microsoft, "SQL Server and ADO.NET," Microsoft Learn. [Online]. Available: <https://learn.microsoft.com/enus/dotnet/framework/data/adonet/sql/>. Accessed: May 15, 2026.
- [8] Microsoft, "Stored Procedures (Database Engine)," Microsoft Learn. [Online]. Available: <https://learn.microsoft.com/en-us/sql/relational-databases/stored-procedures/stored-procedures-database-engine>. Accessed: May 15, 2026.
- [9] Microsoft, "Create a Stored Procedure," Microsoft Learn. [Online]. Available: <https://learn.microsoft.com/enus/sql/relational-databases/stored-procedures/create-a-stored-procedure>. Accessed: May 15, 2026.
- [10] Microsoft, "Transactions (Transact-SQL)," Microsoft Learn. [Online]. Available: <https://learn.microsoft.com/enus/sql/t-sql/language-elements/transactions-transact-sql>. Accessed: May 15, 2026.
- [11] Microsoft, "BEGIN TRANSACTION (Transact-SQL)," Microsoft Learn. [Online]. Available: <https://learn.microsoft.com/en-us/sql/t-sql/language-elements/begin-transaction-transact-sql>. Accessed: May 15, 2026.
- [12] OWASP Foundation, "SQL Injection Prevention Cheat Sheet," OWASP Cheat Sheet Series. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html. Accessed: May 15, 2026.
- [13] OWASP Foundation, "Query Parameterization Cheat Sheet," OWASP Cheat Sheet Series. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Query_Parameterization_Cheat_Sheet.html. Accessed: May 15, 2026.
- [14] OWASP Foundation, "Authentication Cheat Sheet," OWASP Cheat Sheet Series. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html. Accessed: May 15, 2026.
- [15] OWASP Foundation, "Authorization Cheat Sheet," OWASP Cheat Sheet Series. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Authorization_Cheat_Sheet.html. Accessed: May 15, 2026.
- [16] OWASP Foundation, "Logging Cheat Sheet," OWASP Cheat Sheet Series. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Logging_Cheat_Sheet.html. Accessed: May 15, 2026.

Appendices

Appendix A: Screenshot Checklist

Evidence	Inserted?	Notes
Login page	Inserted	Show successful login.
SuperAdmin dashboard/sidebar	Inserted	Show full access.
Staff dashboard/sidebar	Inserted	Show restricted access.
Auditor dashboard/sidebar	Inserted	Show read-only/audit access.
Inventory page	Inserted	Show stock status and item records.
Supplier page	Inserted	Show search and action visibility.
Customer page	Inserted	Show modal and validation.
Purchase order page	Inserted	Show draft/approved purchase.

Purchase items page	Inserted	Show order total and approval.
Sales order page	Inserted	Show create sales modal.
Sales items page	Inserted	Show reserved stock.
Out-of-stock sales item	Inserted	Show Add Item disabled.
Below selling price validation	Inserted	Show error message.
Duplicate item different price	Inserted	Show error message.
Low-stock email	Inserted	Show email alert table.
Returns cascading dropdown	Inserted	Show customer/supplier → order → item flow.
Return maximum quantity	Inserted	Show placeholder with max value.
Stock adjustment modal	Inserted	Show IN/OUT adjustment.
Adjustment OUT error	Inserted	Show negative-stock prevention.
Stock Movement History	Inserted	Show filters and movement records.
System Audit Logs	Inserted	Show filters and export buttons.
Audit CSV/Excel export	Inserted	Show downloaded/exported evidence.

Appendix B – Key Code Listings

Appendix B.1 – Sales Quantity Validation and Draft Reservation

This listing shows how the system checks available stock and existing draft sales quantity before adding an item to a sales order.

Appendix B.2 – Return Quantity Validation

This listing shows how the system prevents sales and purchase returns from exceeding the remaining valid return quantity.

Appendix B.3 – Purchase Approval Stored Procedure

This listing shows the database-level transaction used to approve a purchase, increase inventory quantity and insert a StockLedger record.

Appendix B.4 – Audit Logging with Request Context

This listing shows how the system records user actions with page, IP address, browser, device, event category and severity.

Appendix B.5 – Low-Stock Email Alert Logic

This listing shows how the system builds and sends low-stock alert emails and marks alerts as sent.

Appendix C: Running the Application

This appendix summarises how the system can be run and demonstrated.

1. Open the project in Visual Studio.
2. Restore or create the SQL Server database using the project database script.
3. Update the database connection string in the Web.config file.

4. Configure SMTP email settings if low-stock email alerts are being demonstrated.
5. Build and run the ASP.NET Web Forms application.
6. Log in using a valid account.
7. Demonstrate the four main roles: SuperAdmin, Admin, Staff and Auditor.
8. Demonstrate inventory, supplier and customer management.
9. Create a purchase order, add purchase items and approve the purchase.
10. Create a sales order, add sales items and test validation rules.
11. Approve a sale and verify that stock movement is recorded.
12. Create a sales return and purchase return.
13. Create stock adjustment IN and OUT records.
14. Review Stock Movement History.
15. Review System Audit Logs.
16. Export audit logs to CSV or Excel.